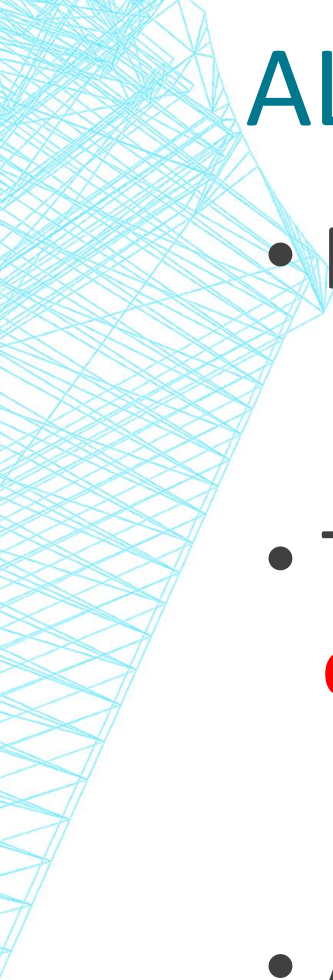


ALGORITMOS GULOSOS

João Pedro Oliveira Batisteli





ALGORITMOS GULOSOS

- No original, *greedy algorithms*.
- Tipicamente utilizados para resolver **problemas de otimização**.
- Aplicação mais adequada em problemas que possuem *subestrutura ótima*.

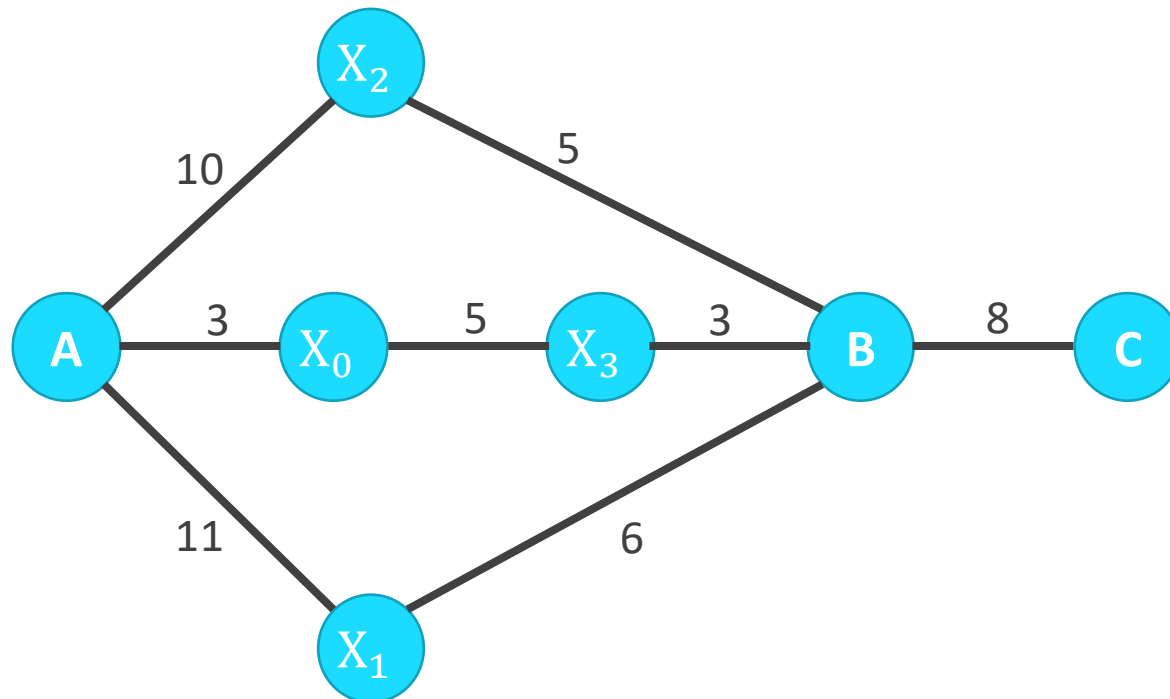


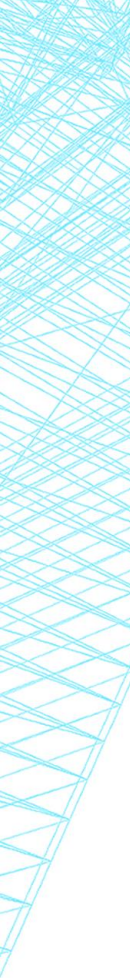
SUBESTRUTURA ÓTIMA

- Acontece quando uma solução ótima para o problema contém em seu interior soluções ótimas para subproblemas.

EXEMPLO – CAMINHO MAIS CURTO

- Imagine que você está viajando de carro da **Cidade A** para a **Cidade C**, e a rota mais rápida e otimizada (a solução ótima) passa por uma cidade intermediária, a **Cidade B**.



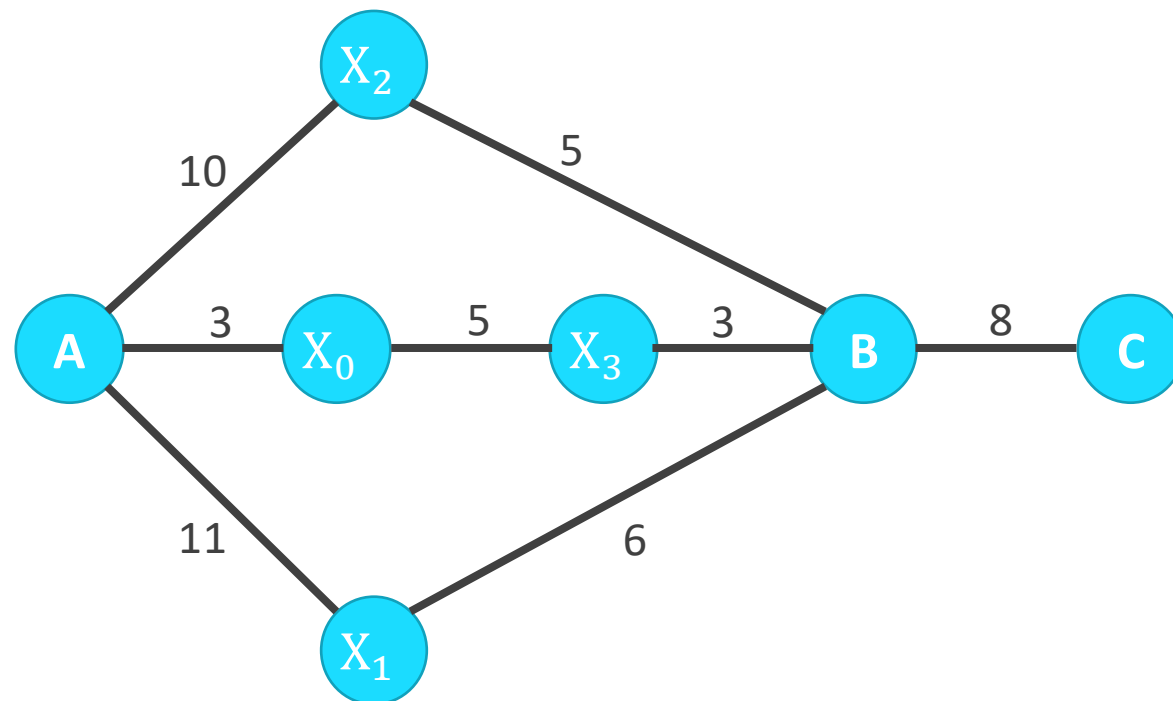


EX

1)

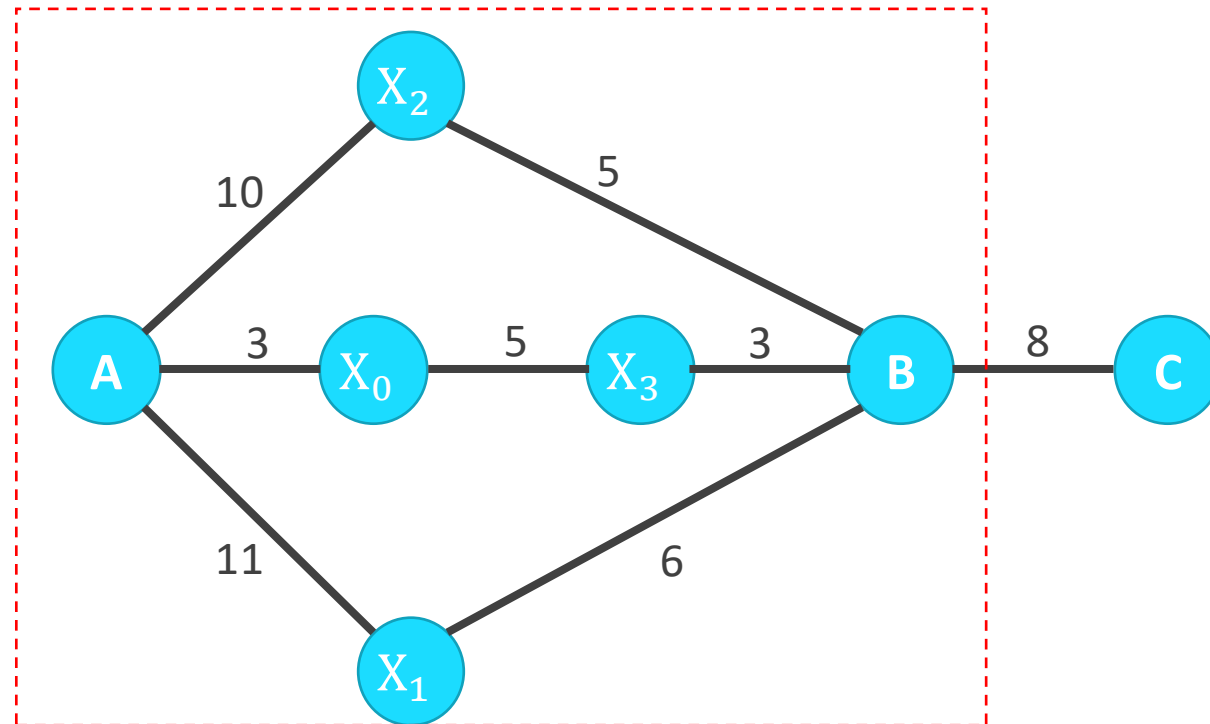
ca

1) O trajeto da **Cidade A** até a **Cidade B** *precisa, obrigatoriamente*, ser o caminho **mais rápido possível entre essas duas cidades** (o subproblema).



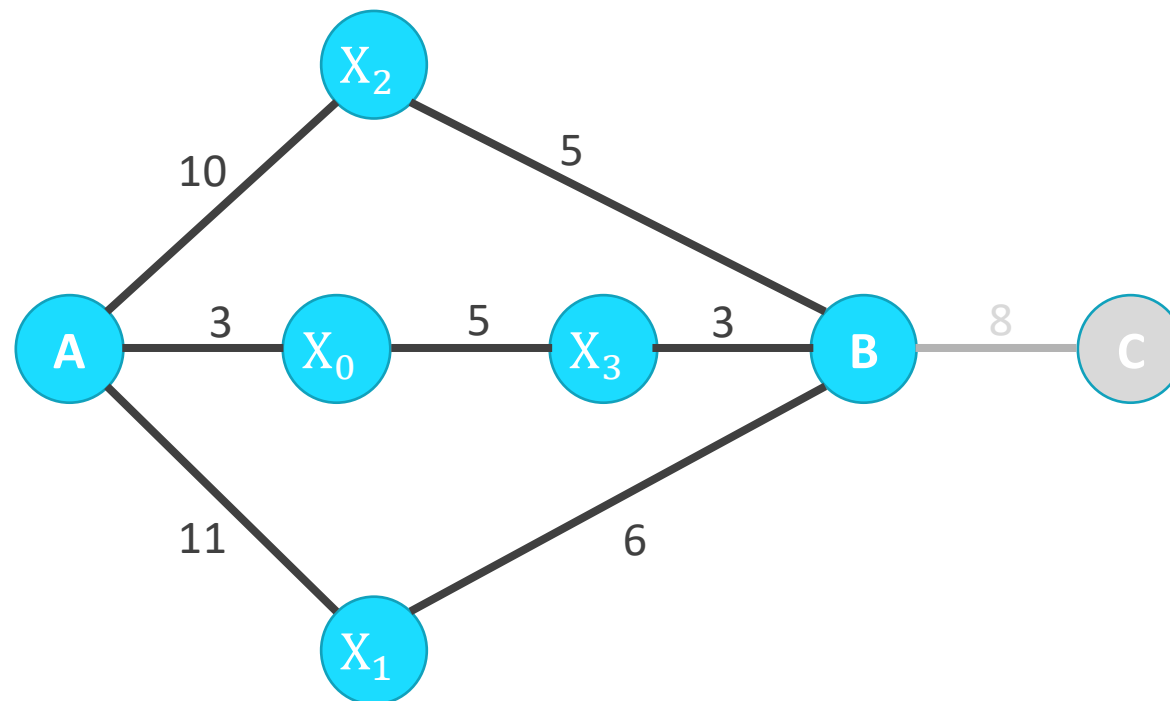
EXEMPLO – CAMINHO MAIS CURTO

1) O trajeto da **Cidade A** até a **Cidade B** *precisa, obrigatoriamente*, ser o caminho **mais rápido possível entre essas duas cidades** (o subproblema).



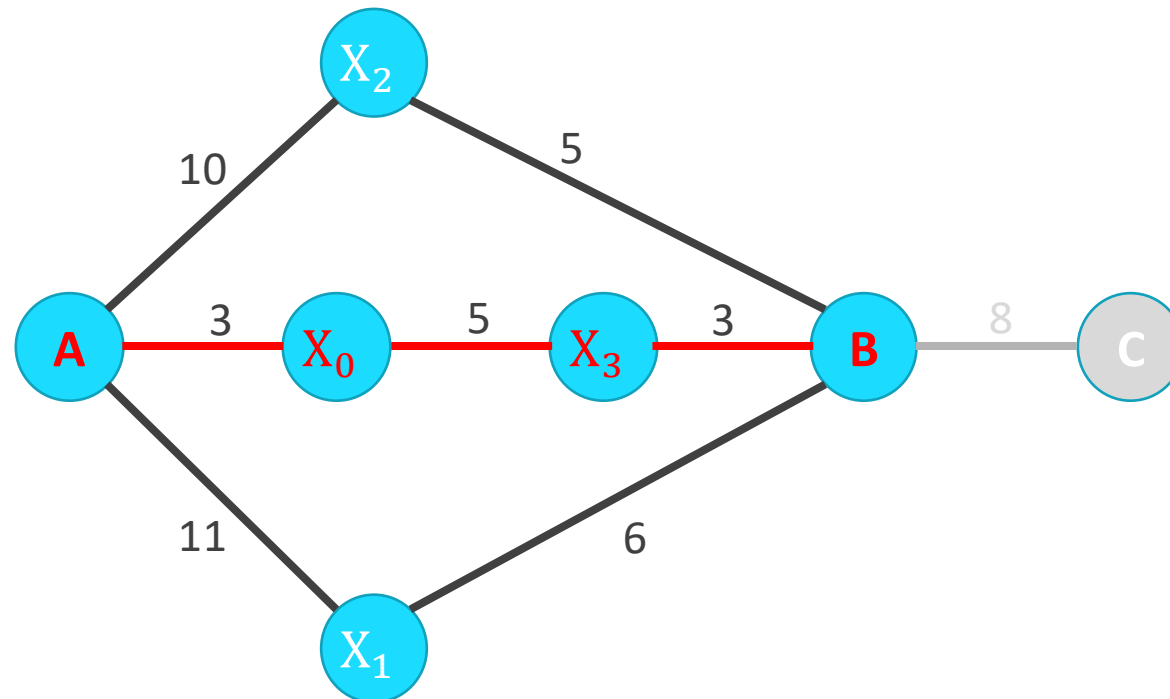
EXEMPLO – CAMINHO MAIS CURTO

1) O trajeto da **Cidade A** até a **Cidade B** *precisa, obrigatoriamente*, ser o caminho **mais rápido possível entre essas duas cidades** (o subproblema).



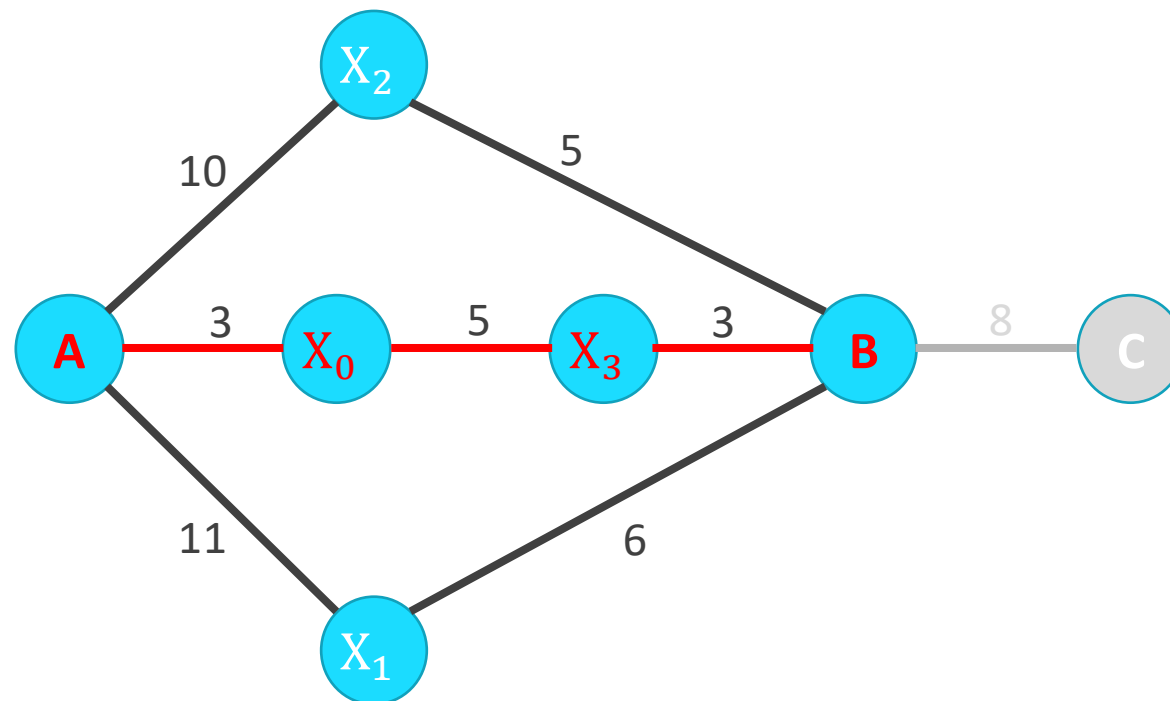
EXEMPLO – CAMINHO MAIS CURTO

1) O trajeto da **Cidade A** até a **Cidade B** *precisa, obrigatoriamente*, ser o caminho **mais rápido possível entre essas duas cidades** (o subproblema).



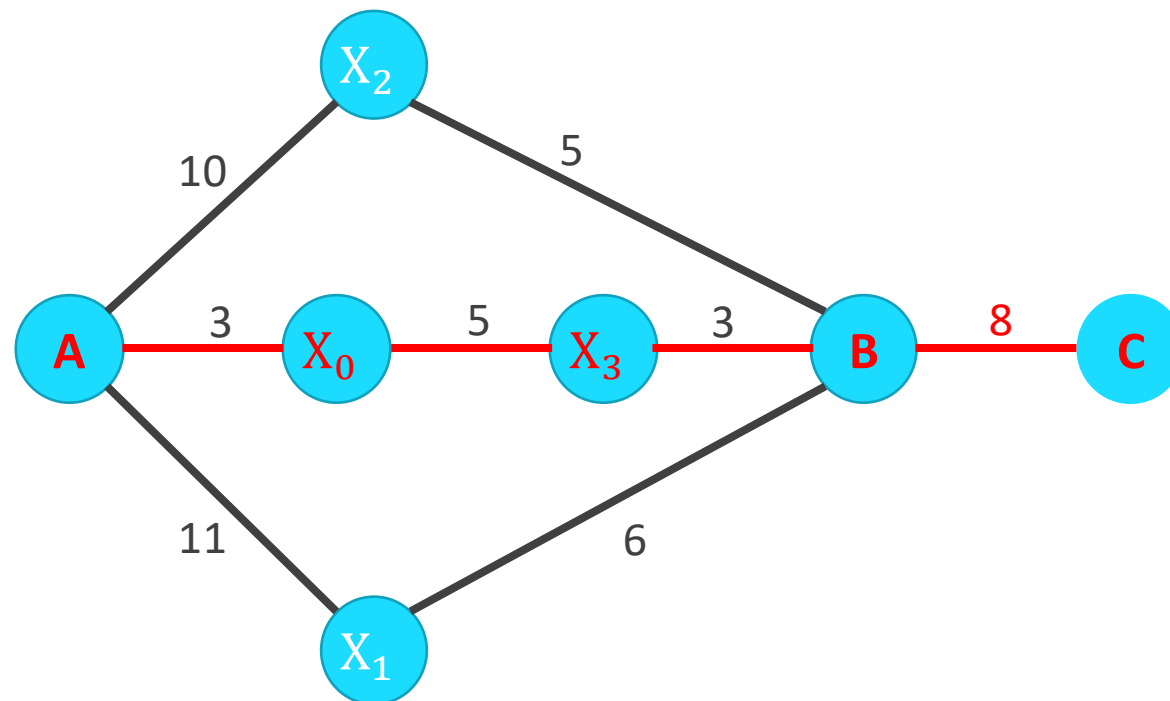
EXEMPLO – CAMINHO MAIS CURTO

2) Se existisse um atalho mais rápido entre A e B que não foi usado, nós poderíamos ter pego esse atalho. Isso faria com que o tempo total da viagem inteira (de A até C) fosse ainda menor! Como partimos da premissa de que a viagem completa já era a mais rápida possível, é impossível que exista um subcaminho melhor.



EXEMPLO – CAMINHO MAIS CURTO

3) Portanto, a solução ótima do problema maior (A até C) contém em seu interior a solução ótima do subproblema (A até B).



PROBLEMAS DE OTIMIZAÇÃO

- Algoritmos para problemas de otimização, normalmente passam por uma sequência de etapas e cada etapa tem um conjunto de escolhas.
- Um ***algoritmo guloso*** sempre faz a escolha que parece ser a melhor no **momento em questão**.
 - Isto é, faz uma escolha localmente ótima, na esperança de que essa escolha leve a uma solução globalmente ótima.

ALGORITMOS GULOSOS

- Um ***algoritmo guloso*** sempre faz a escolha que parece ser a melhor no **momento em questão**.
 - Isto é, faz uma escolha localmente ótima, na esperança de que essa escolha leve a uma solução globalmente ótima.
- Definição do ***critério guloso***.

CRITÉRIO GULOSO

- Verifica-se o conjunto de soluções produzidas pelos candidatos atuais (sem ter uma visão de todo o problema).
- Seleciona, no conjunto, o candidato mais promissor (ou melhor):
 - **Decisão local.**

ALGORITMOS GULOSOS

- A função de seleção pode ser maximizadora ou minimizadora.
 - Max: candidato que proporcione o maior ganho local.
 - Min: candidato que proporcione o menor custo local.

ALGORITMOS GULOSOS

Algoritmo geral:

- 1) Listar todos os candidatos no passo atual;
- 2) Usando o critério guloso, selecionar o melhor local;
- 3) Adicionar o melhor local na solução e retirá-lo do conjunto de candidatos;
- 4) Repetir até chegar a uma solução.

ALGORITMOS GULOSOS

```
Função AlgoritmoGulosoGeral(Candidatos)
// Inicializa a solução final como vazia
Solucao <- {}

//Repetir até chegar a uma solução (ou acabarem as opções)
Enquanto (Solucao não estiver completa) e (Candidatos não estiver vazio)faça:

    //Listar candidatos e, usando o critério guloso, selecionar o melhor
    MelhorCandidato <- SelecionarMelhor(Candidatos)

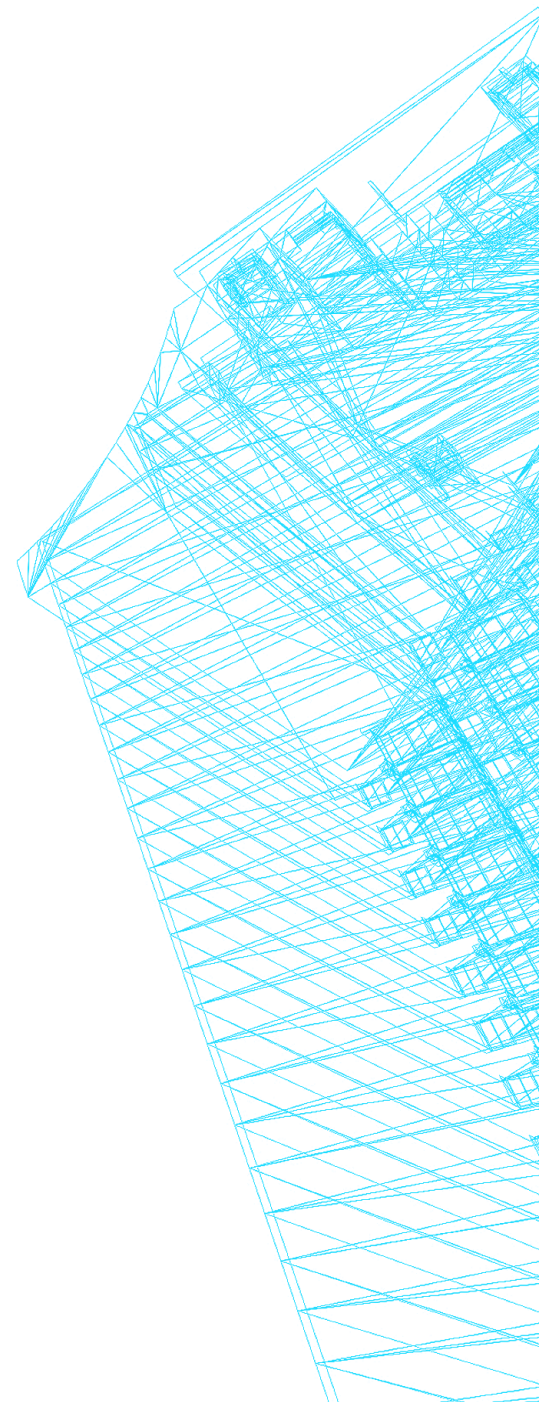
    //Adicionar o melhor local/candidato na solução
    Solucao <- Solucao + {MelhorCandidato}

    //Retirá-lo do conjunto de candidatos
    Candidatos <- Candidatos - {MelhorCandidato}

Fim Enquanto

Retornar Solucao
Fim Função
```

EXEMPLOS



CAIXEIRO VIAJANTE

- Um caixeiro viajante deseja visitar N cidades e entre cada par de cidades existe uma rota;
- Cada rota possui uma distância ou custo necessários para percorrê-la;
- O caixeiro viajante deseja encontrar:
 - um caminho que passe por cada cidade apenas uma vez e retorne à origem, com o menor custo possível.

CAIXEIRO VIAJANTE

- Força bruta:
 - 1) montamos todas as rotas possíveis,
 - 2) calculamos o comprimento de cada uma delas,
 - 3) selecionamos a melhor rota.



CAIXEIRO VIAJANTE (GULOSO)

1. Listar todos os candidatos no passo atual;
2. Usando o critério guloso, selecionar o melhor local;
3. Adicionar o melhor local na solução e retirá-lo do conjunto de candidatos;
4. Repetir até chegar a uma solução.



CAIXEIRO VIAJANTE (GULOSO)

1. Listar todos os candidatos no passo atual;
2. Usando o **critério guloso**, selecionar o melhor local;
3. Adicionar o melhor local na solução e retirá-lo do conjunto de candidatos;
4. Repetir até chegar a uma solução.

CAIXEIRO VIAJANTE (GULOSO)

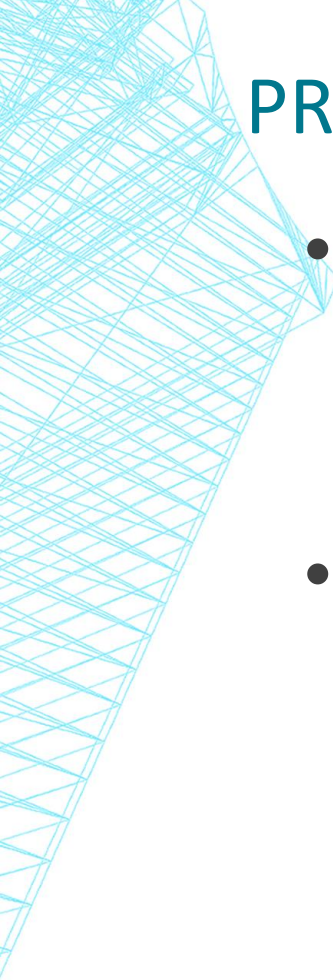
1. Listar todos os candidatos no passo atual;
2. Usando o **critério guloso**, selecionar o melhor local;
3. Adicionar o melhor local na solução e retirá-lo do conjunto de candidatos;
4. Repetir até chegar a uma solução.

Critério guloso: *vizinho não visitado mais próximo.*

CAIXEIRO VIAJANTE (GULOSO)

1. Listar todos os candidatos no passo atual;
2. Usando o critério guloso, selecionar o melhor local;
3. Adicionar o melhor local na solução e retirá-lo do conjunto de candidatos;
4. Repetir até chegar a uma solução.

-
1. Determine as cidades não visitadas;
 2. Selecione a menor rota até uma cidade não visitada;
 3. Inclua a rota e a cidade na solução e marque a visita;
 4. Repita até todas as cidades terem sido visitadas.



PROBLEMAS DE OTIMIZAÇÃO: O QUE ESTAMOS BUSCANDO?

- Em Ciência da Computação, um **Problema de Otimização** exige encontrar a *melhor* solução possível entre um conjunto de soluções válidas.
- Para resolvermos o problema, precisamos modelar dois elementos centrais:
 - **Função Objetivo:** É a métrica matemática que queremos otimizar. O nosso foco sempre será **maximizar** (ex: lucro, eficiência) ou **minimizar** (ex: custo, tempo, distância) esta função.
 - **Restrições:** São as "regras do jogo". **Condições inflexíveis** que a solução precisa **obrigatoriamente satisfazer para ser considerada uma solução válida**.

CAIXEIRO VIAJANTE

Como modelamos o clássico problema de um vendedor que precisa visitar um conjunto de cidades de forma eficiente?

- **A Função Objetivo:**
 - *Minimizar a distância total (ou custo) do percurso completo.*
- **As Restrições (As regras do jogo):**
 - O vendedor deve visitar cada cidade exatamente uma única vez.
 - O trajeto deve começar e terminar obrigatoriamente na mesma cidade de origem

A ABORDAGEM GULOSA NO CAIXEIRO VIAJANTE

- Como aplicar a estratégia gulosa para tentar minimizar a nossa Função Objetivo?
- **O Critério Guloso na prática:**
 - A cada **iteração**, o caixeiro olha para as cidades disponíveis e simplesmente viaja para a cidade vizinha *mais próxima* (menor custo imediato).
- **A armadilha da escolha local:**
 - O algoritmo guloso busca o ótimo local a cada passo, **mas ignora o panorama geral**.
 - Fazer a melhor **escolha agora pode obrigar o caixeiro a pegar um caminho gigantesco no final** apenas para satisfazer a restrição de voltar à origem.

CAIXEIRO VIAJANTE - ALGORITMO

Entradas:

$C = \{c_0, c_1, c_2, \dots, c_n\}$, conjunto de cidades

$D = \{d_{0,1}, d_{0,2}, \dots, d_{n-1,n}\}$, conjunto de distâncias

CAIXEIRO VIAJANTE - ALGORITMO

```
Função CaixeiroViajanteGuloso(C, D)
  cidadeInicial <- EscolhaDoUsuario()
  ci <- cidadeInicial
  Caminho <- {ci}
  DistanciaTotal <- 0

  // Essencial: marcar a origem como visitada para não voltar nela indevidamente
  MarcarComoVisitada(ci)

  // 2. Iteração para visitar todas as cidades restantes
  Enquanto (houver cidades não visitadas em C) faça:

    // Critério Guloso: seleciona a vizinha disponível de menor custo
    cj <- SelecioneCidadeNaoVisitadaMaisProximaDe(ci, D)

    // Atualiza a solução (Caminho e Função Objetivo)
    Caminho <- Caminho + {cj}
    DistanciaTotal <- DistanciaTotal + Distancia(ci, cj)

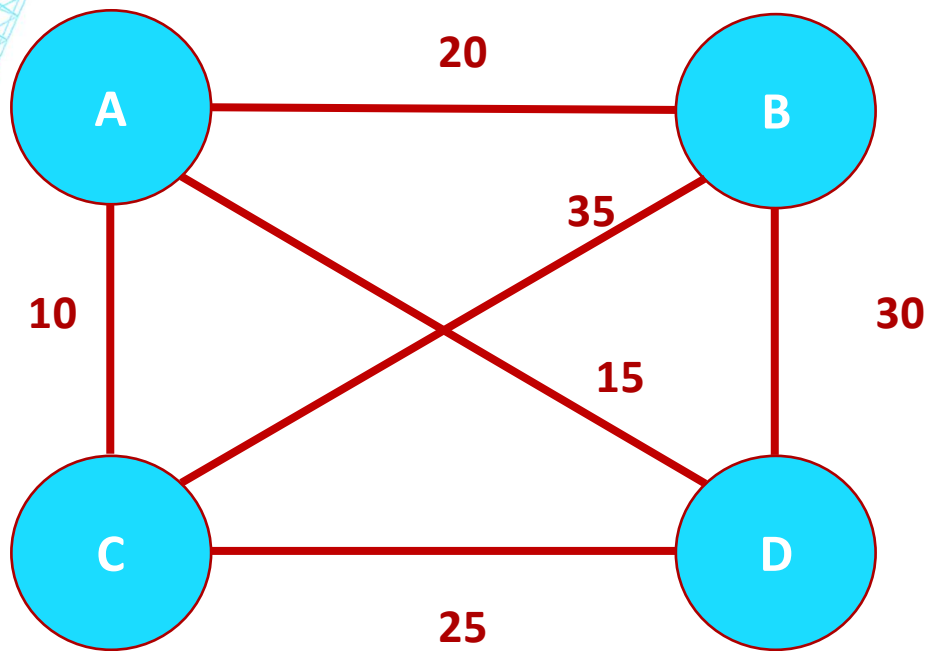
    // Atualiza as restrições e o estado do caixeiro
    MarcarComoVisitada(cj)
    ci <- cj

  Fim Enquanto

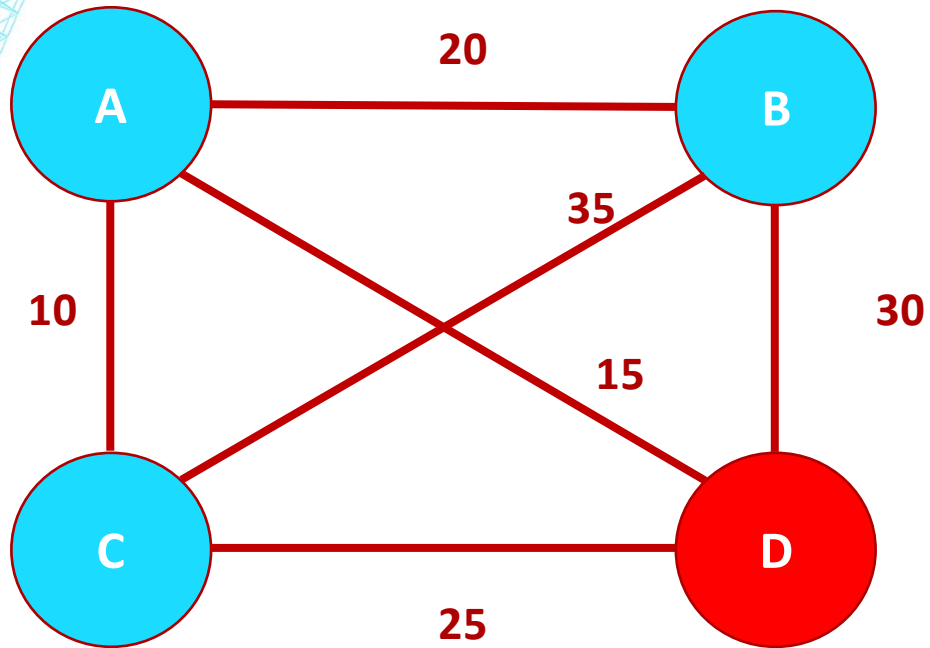
  // 3. Fechamento do ciclo (Restrição do Caixeiro Viajante)
  Caminho <- Caminho + {cidadeInicial}
  DistanciaTotal <- DistanciaTotal + Distancia(ci, cidadeInicial)

  Retornar Caminho, DistanciaTotal
Fim Função
```

CAIXEIRO VIAJANTE



CAIXEIRO VIAJANTE

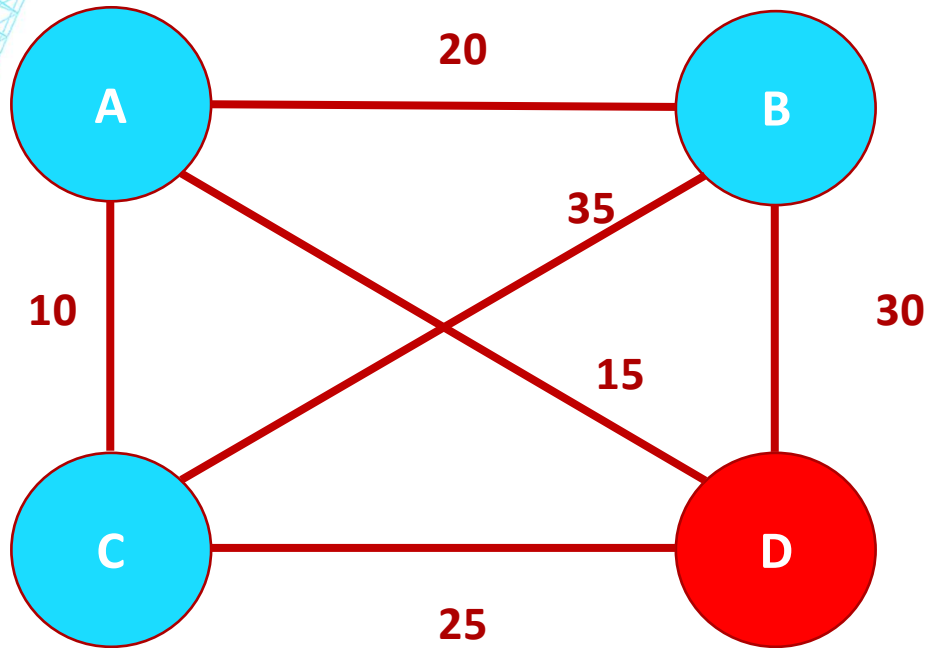


Cidade inicial = D

Caminho = {D}

Distância = 0

CAIXEIRO VIAJANTE



D->A = custo 15

D->B = custo 30

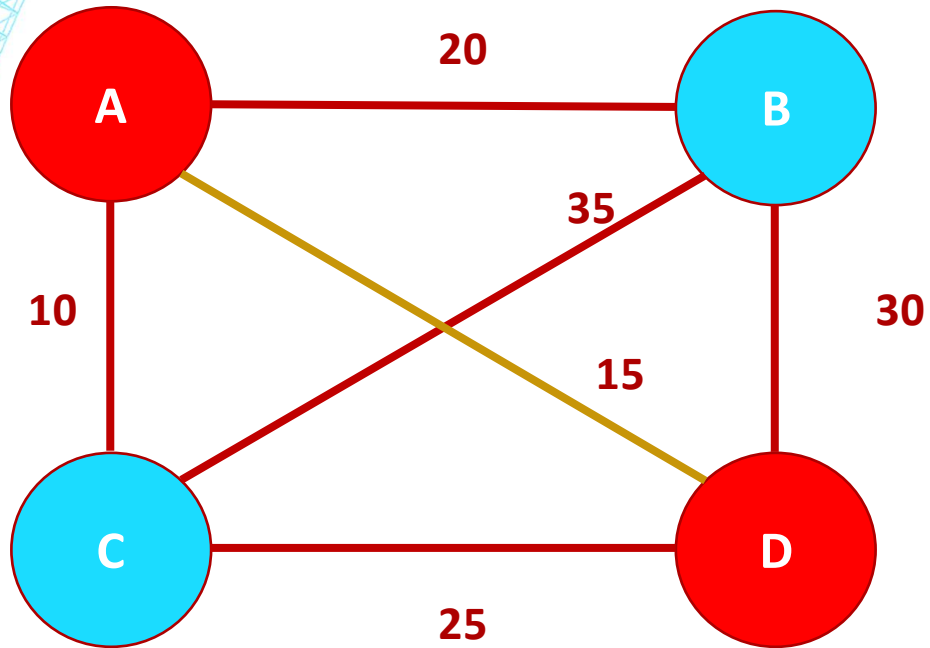
D->C = custo 25

Cidade inicial = D

Caminho = {D}

Distância = 0

CAIXEIRO VIAJANTE



D->A = custo 15

D->B = custo 30

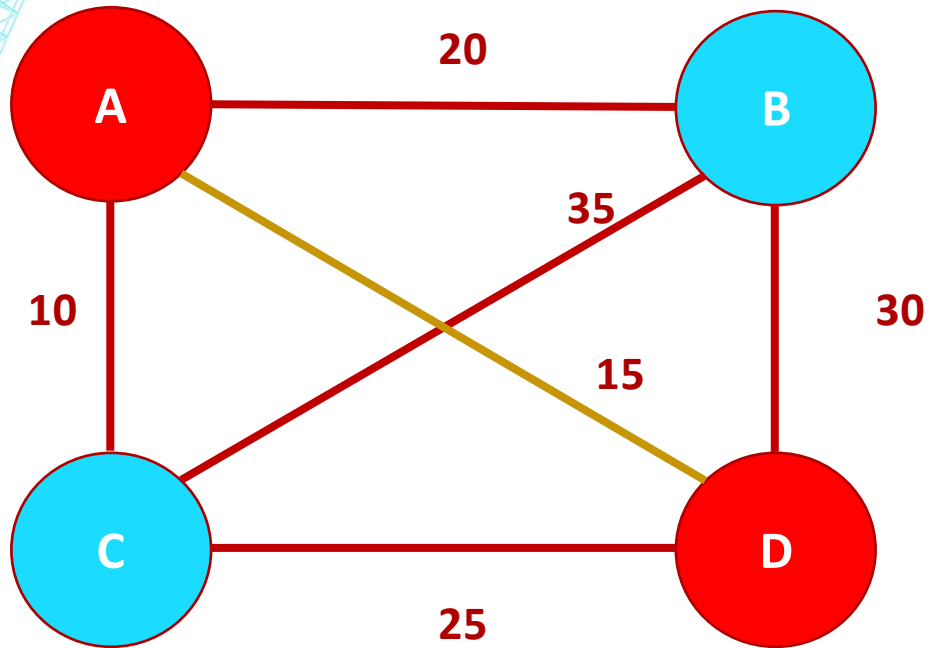
D->C = custo 25

Cidade inicial = D

Caminho = {D}+{A}

Distância = 0+15

CAIXEIRO VIAJANTE

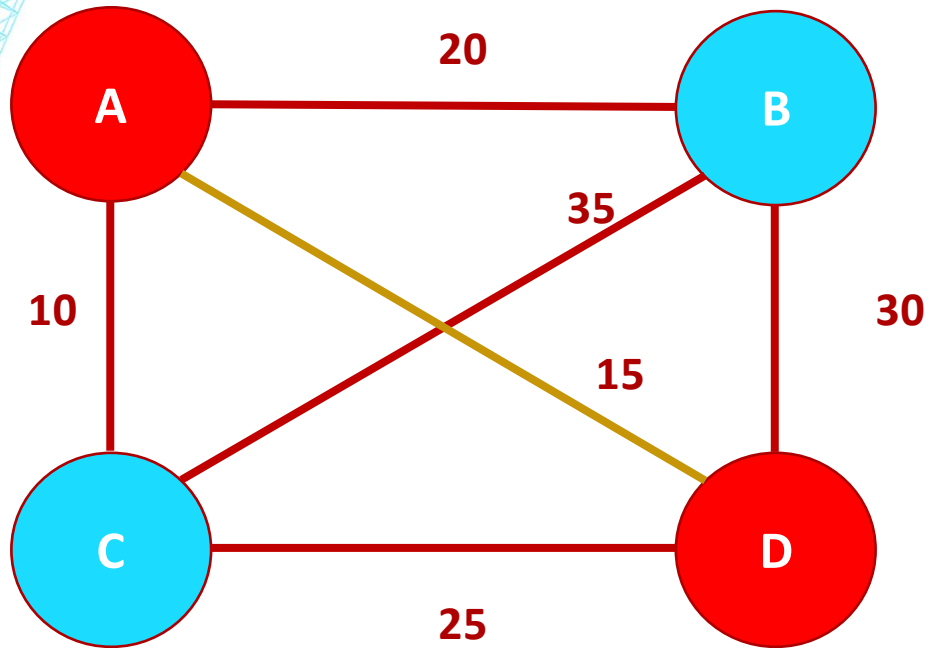


Cidade inicial = D

Caminho = {D,A}

Distância = 15

CAIXEIRO VIAJANTE



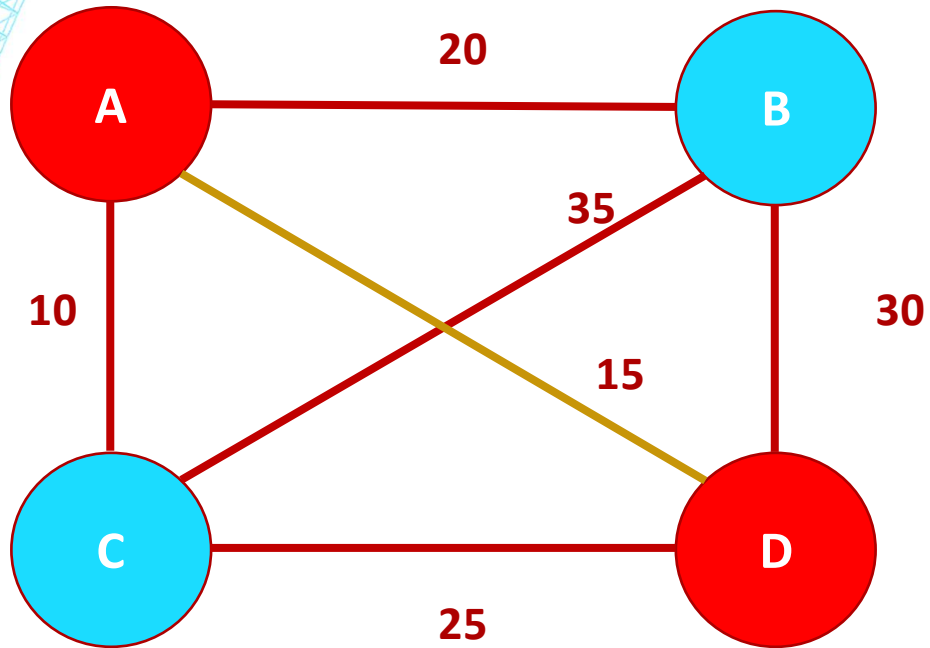
Cidade inicial = D

Caminho = {D,A}

Distância = 15

Cidade atual = A

CAIXEIRO VIAJANTE



A->C = custo 10

A->B = custo 20

A->D = já visitado

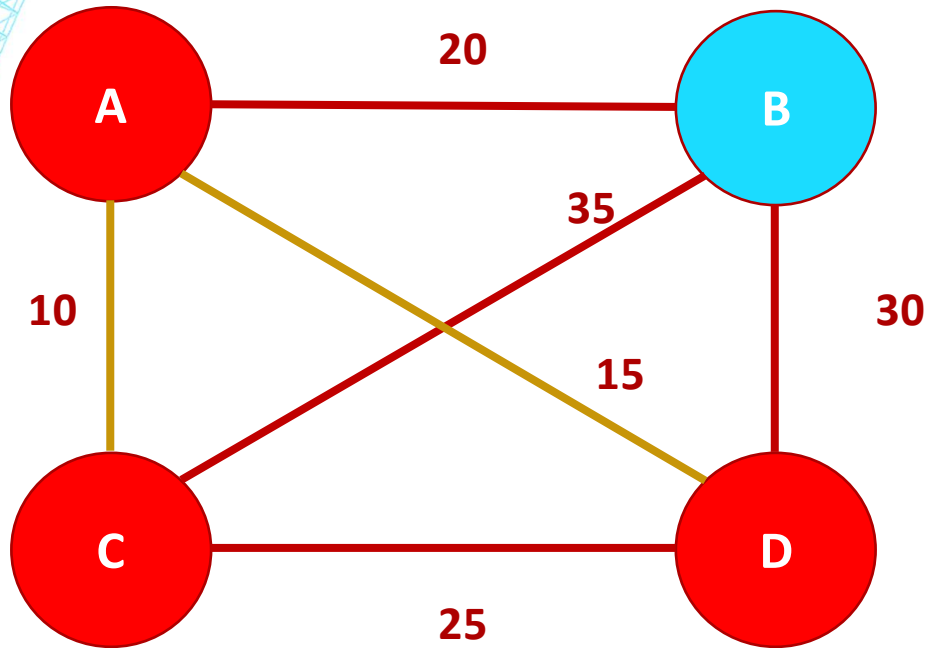
Cidade inicial = D

Caminho = {D,A}

Distância = 15

Cidade atual = A

CAIXEIRO VIAJANTE



A->C = custo 10

A->B = custo 20

A->D = já visitado

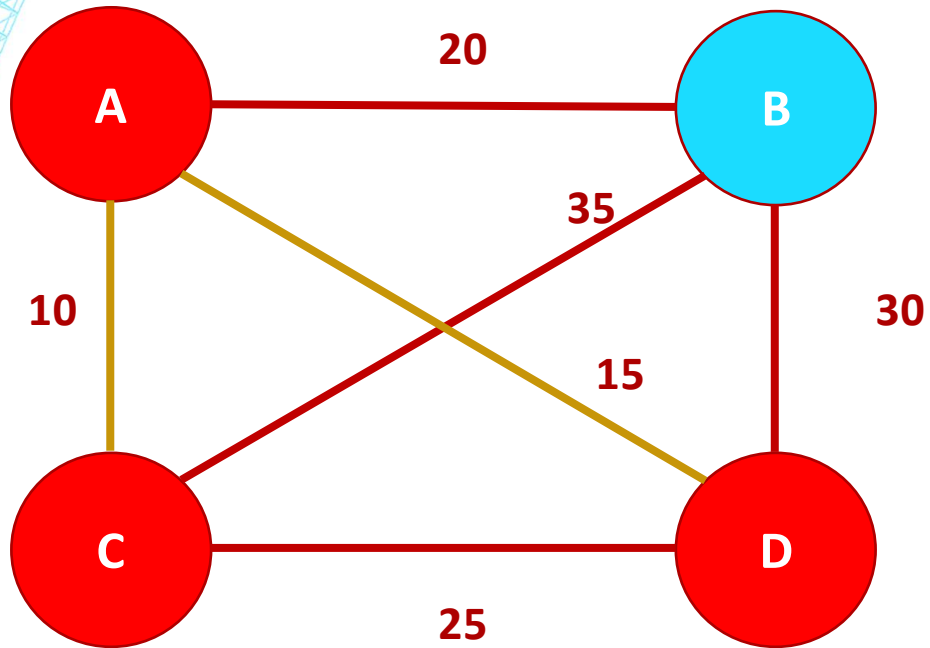
Cidade inicial = D

Caminho = {D,A}+{C}

Distância = 15+10

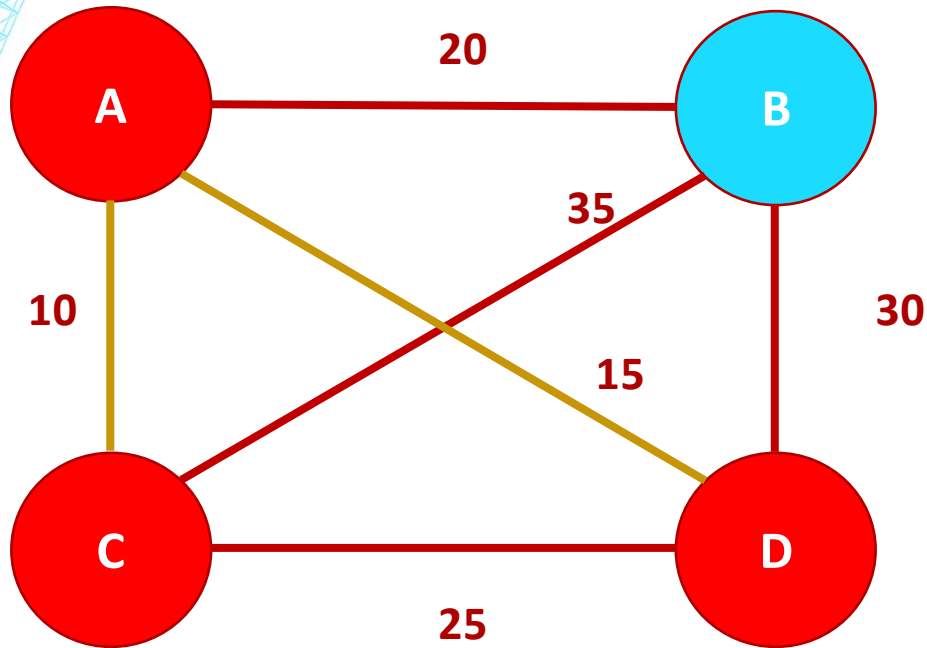
Cidade atual = A

CAIXEIRO VIAJANTE



Cidade inicial = D
Caminho = {D,A,C}
Distância = 25
Cidade atual = C

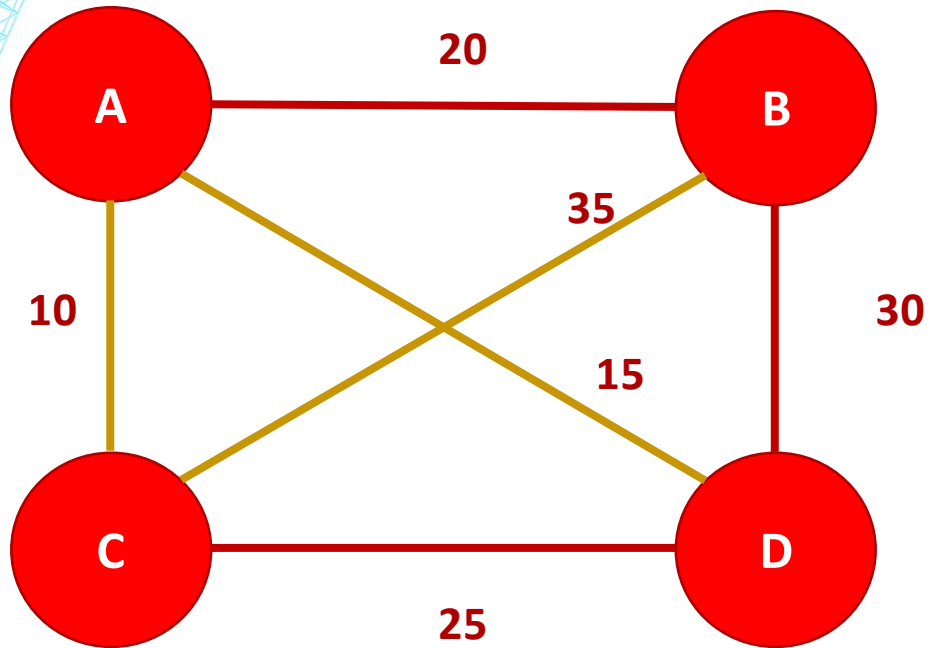
CAIXEIRO VIAJANTE



Cidade inicial = D
Caminho = {D,A,C}
Distância = 25
Cidade atual = C

C->B = custo 35
C->D = D já visitado
C->A = A já visitado

CAIXEIRO VIAJANTE



C->B = custo 35

C->D = D já visitado

C->A = A já visitado

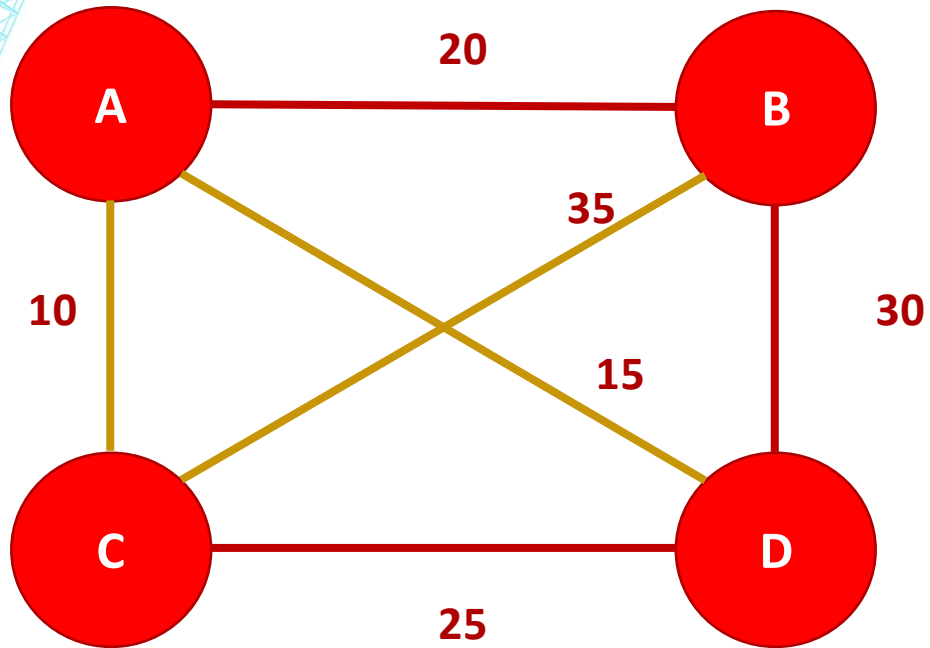
Cidade inicial = D

Caminho = {D,A,C}+{B}

Distância = 25+35

Cidade atual = C

CAIXEIRO VIAJANTE



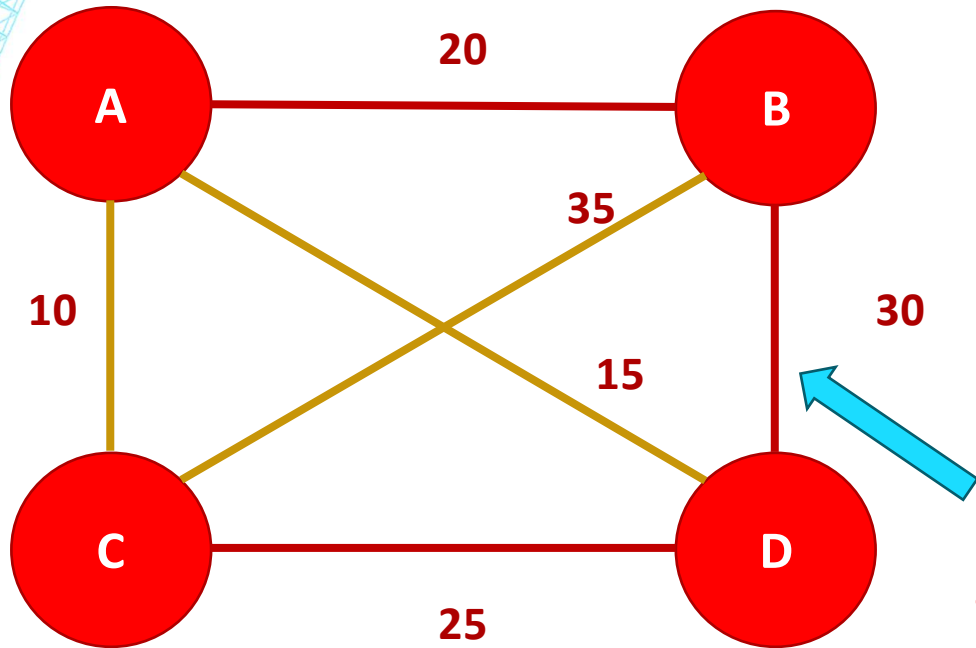
Cidade inicial = D

Caminho = {D,A,C,B}

Distância = 60

Cidade atual = B

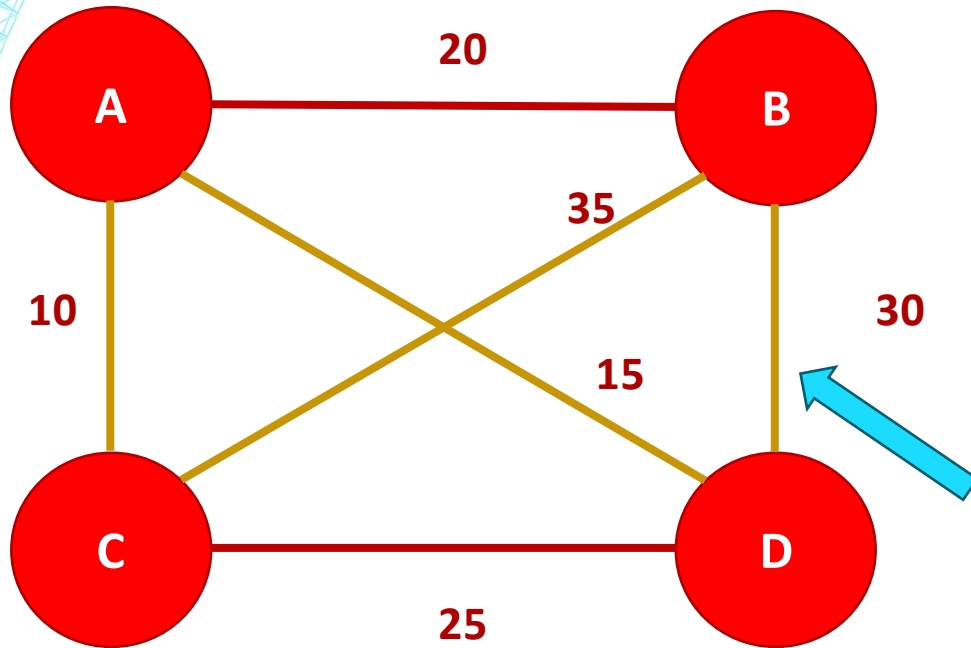
CAIXEIRO VIAJANTE



Cidade inicial = D
Caminho = {D,A,C,B}
Distância = 60
Cidade atual = B

Já visitamos todos os vértices/Cidades,
nos resta tentar voltar para a cidade
inicial.

CAIXEIRO VIAJANTE



Cidade inicial = D

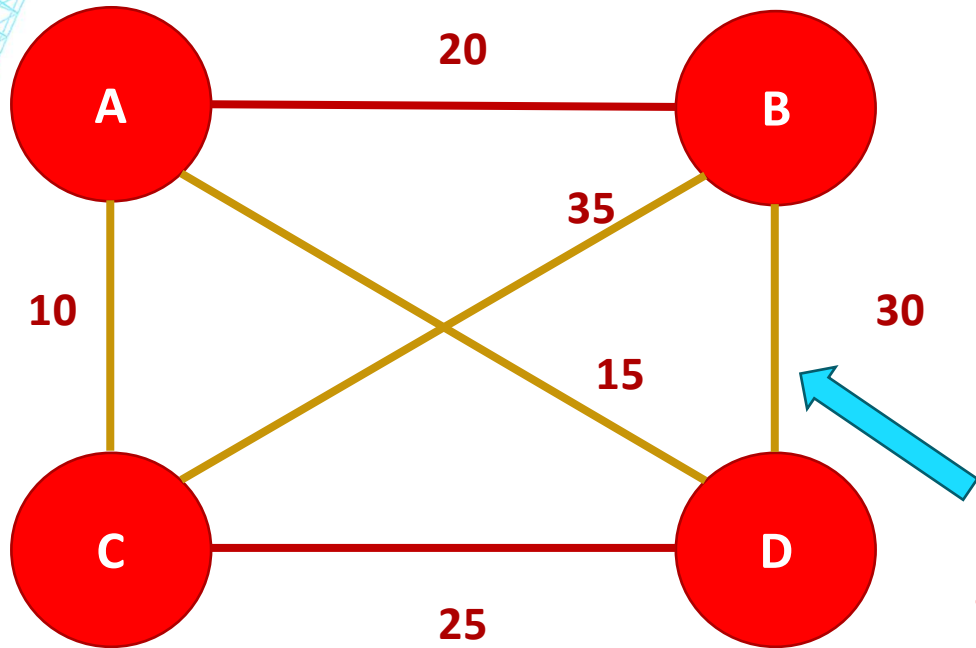
Caminho = {D,A,C,B}+{D}

Distância = 60+30

Cidade atual = B

Já visitamos todos os vértices/Cidades,
nos resta tentar voltar para a cidade
inicial.

CAIXEIRO VIAJANTE



Cidade inicial = D

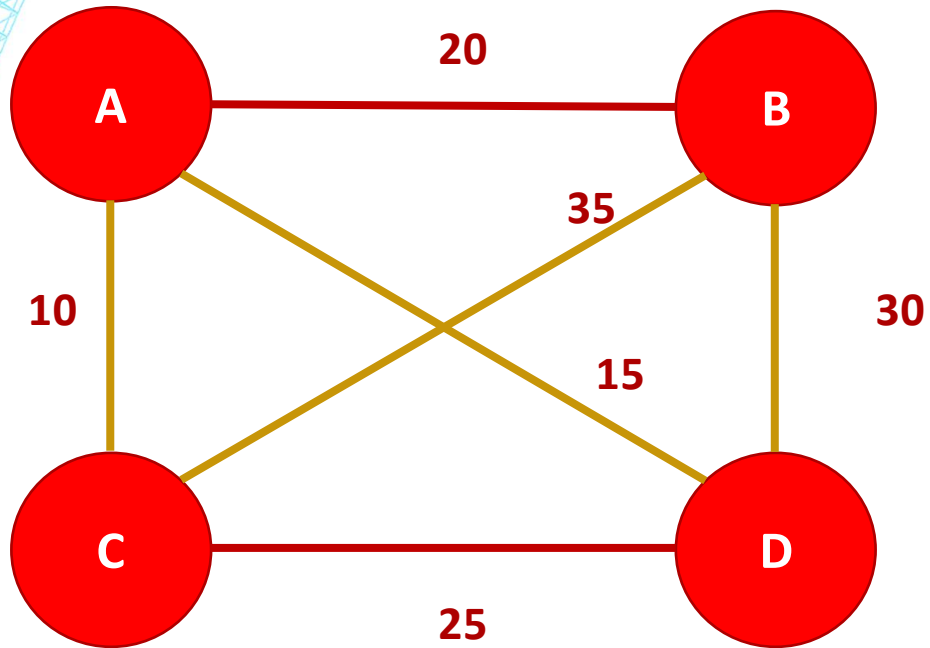
Caminho = {D,A,C,B,D}

Distância = 90

Cidade atual = D

Já visitamos todos os vértices/Cidades,
nos resta tentar voltar para a cidade
inicial.

CAIXEIRO VIAJANTE

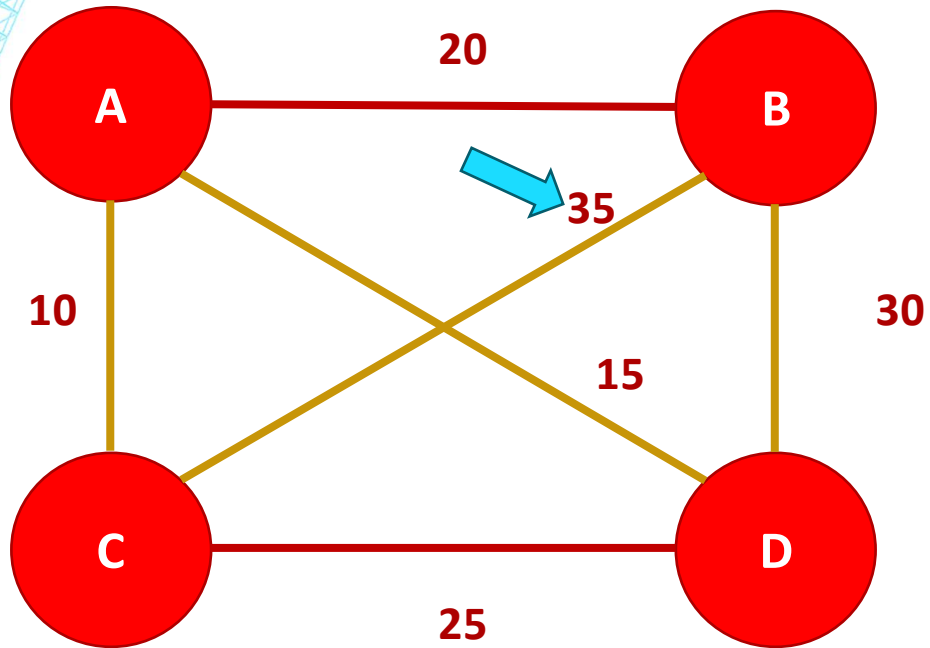


É a melhor solução?

Caminho = {D,A,C,B,D}

Distância = 90

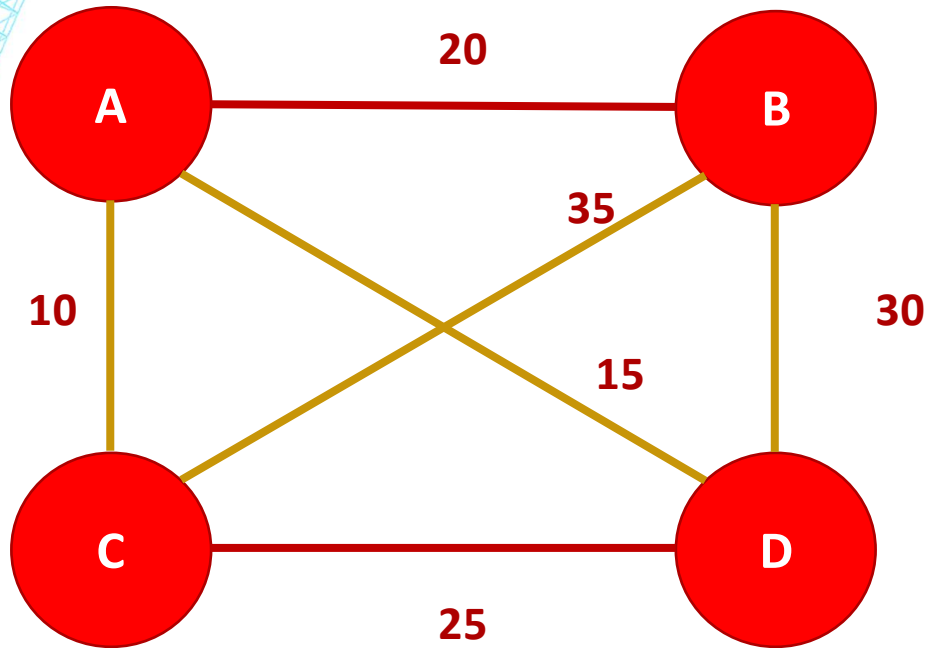
CAIXEIRO VIAJANTE



Problema

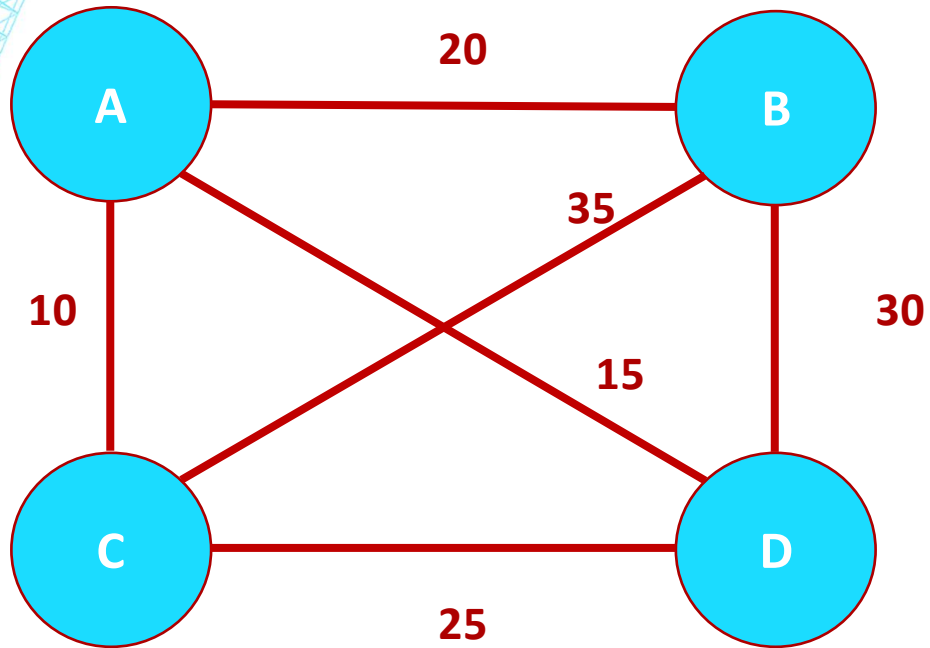
Se a distância até a cidade restante for muito grande, a solução é obrigada a escolher esse caminho.

CAIXEIRO VIAJANTE

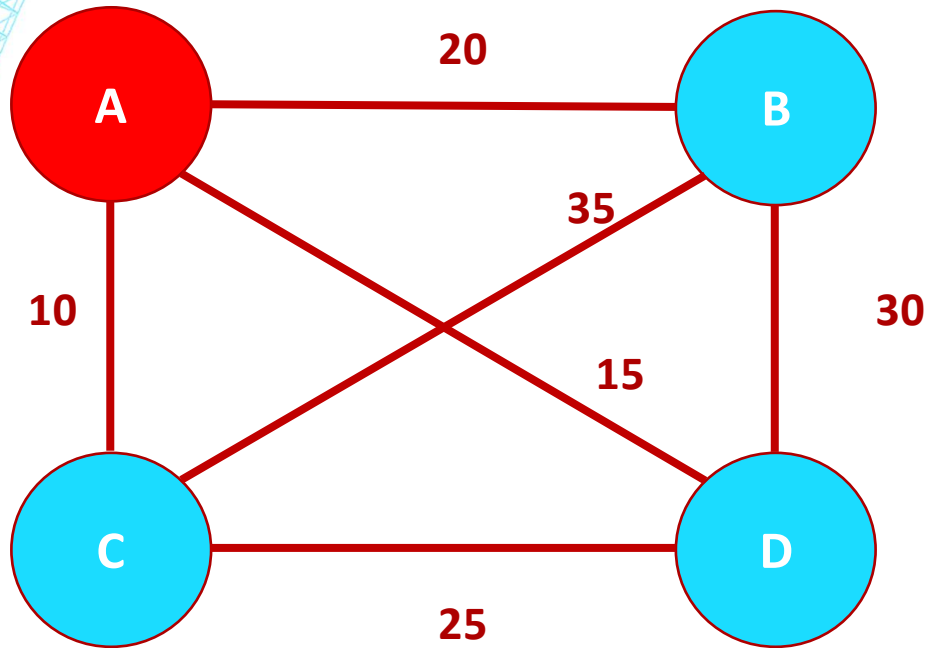


Ótimo local
 $\neq$
Ótimo global

CAIXEIRO VIAJANTE – EX2



CAIXEIRO VIAJANTE – EX2

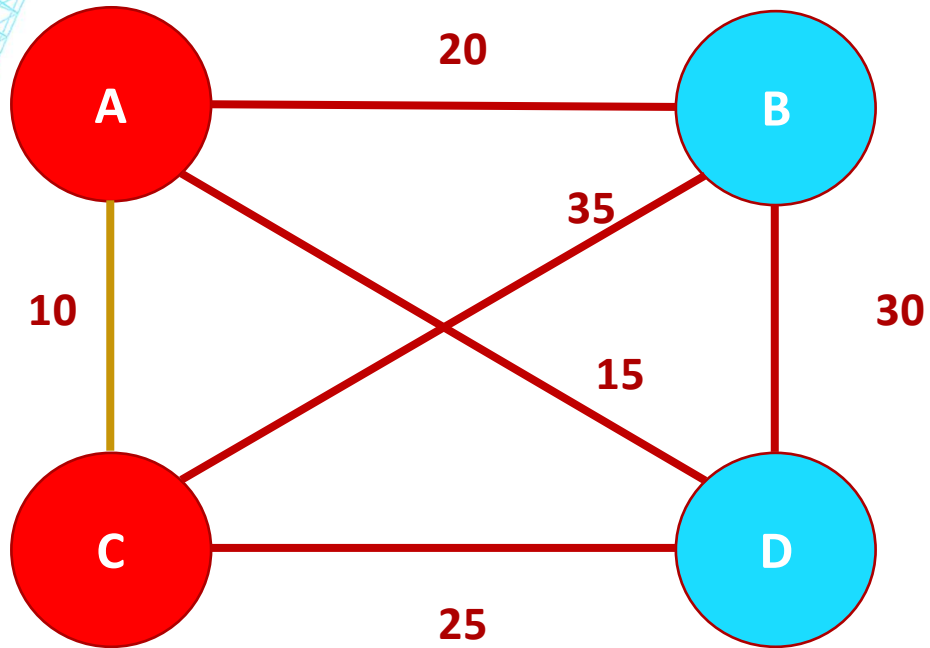


Cidade inicial = A

Caminho = {A}

Distância = 0

CAIXEIRO VIAJANTE – EX2



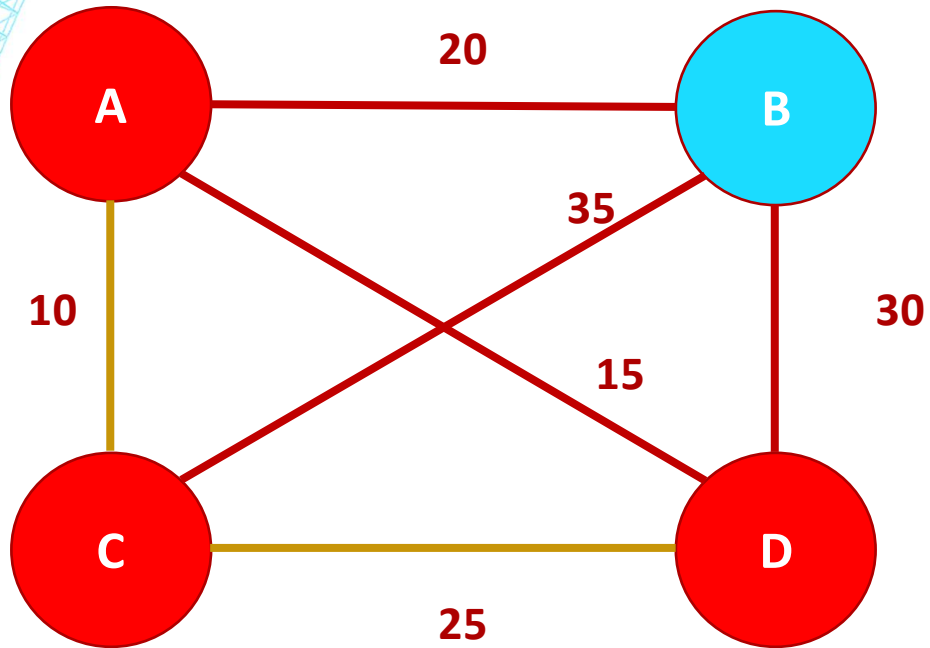
Cidade inicial = A

Caminho = {A,C}

Distância = 10

Cidade atual = C

CAIXEIRO VIAJANTE – EX2



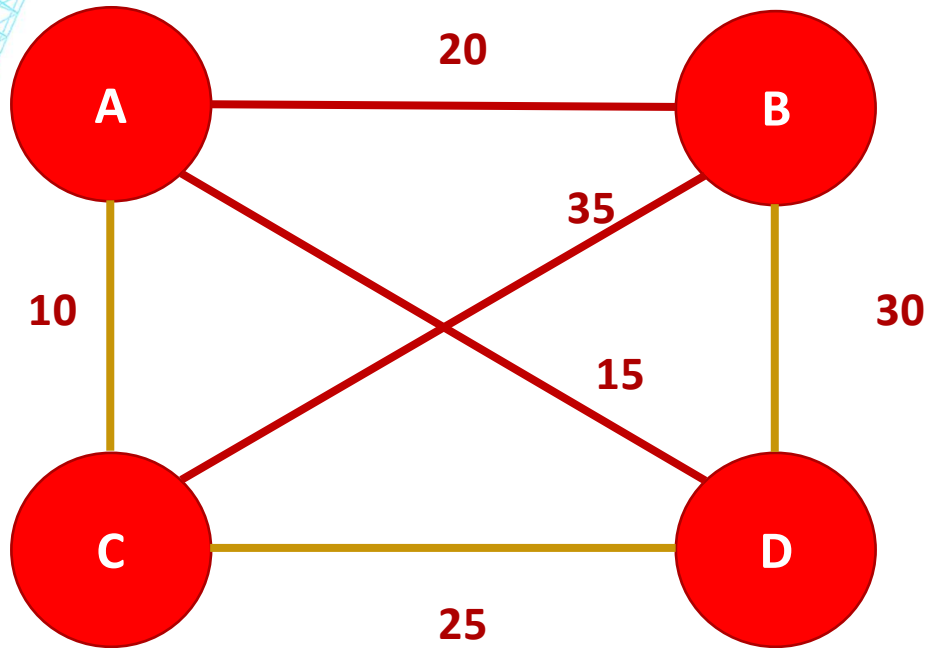
Cidade inicial = A

Caminho = {A,C,D}

Distância = 35

Cidade atual = C

CAIXEIRO VIAJANTE – EX2



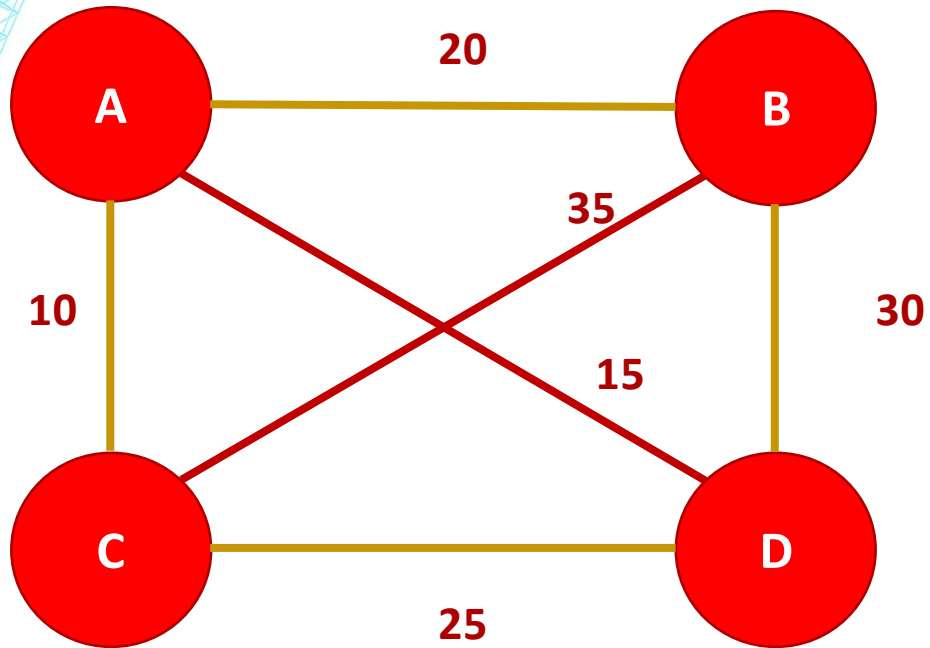
Cidade inicial = A

Caminho = {A,C,D,B}

Distância = 65

Cidade atual = B

CAIXEIRO VIAJANTE – EX2



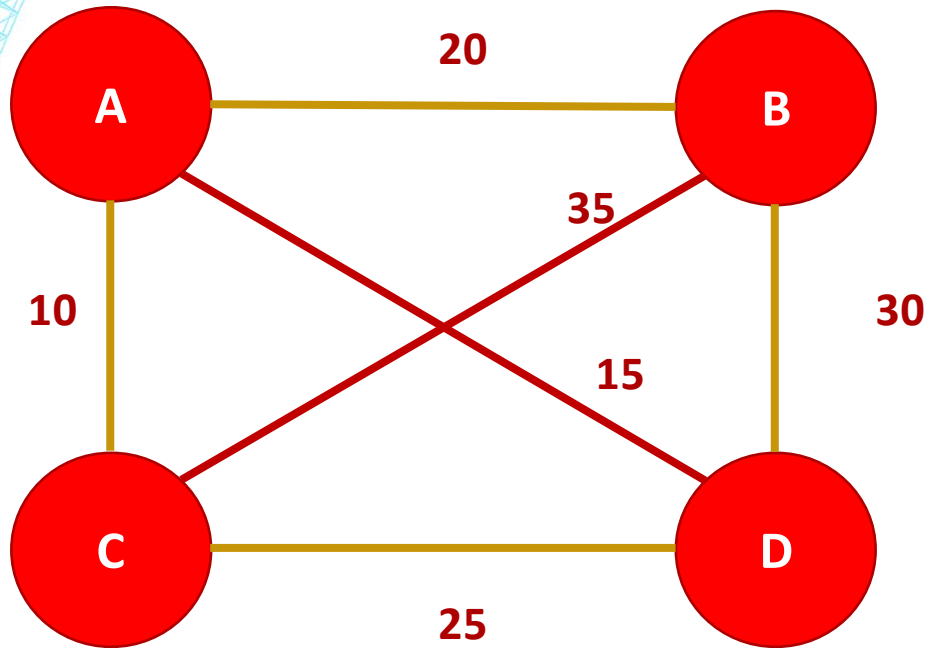
Cidade inicial = A

Caminho = {A,C,D,B,A}

Distância = 85

Cidade atual = A

CAIXEIRO VIAJANTE – EX2



Caminho = {A,C,D,B,A}
Distância = 85

Caminho = {D,A,C,B,D}
Distância = 90

O PROBLEMA DA MOCHILA *(KNAPSACK PROBLEM)*

- Objetivo: Maximizar – conjunto de itens com o maior valor somado possível, sem exceder a capacidade.
- Critério guloso: ?

O PROBLEMA DA MOCHILA (*KNAPSACK PROBLEM*)

- Objetivo: Maximizar – conjunto de itens com o maior valor somado possível, sem exceder a capacidade.
- Critério guloso- Alternativa 1:
 - selecionar o item individual ainda não escolhido que caiba na mochila e **tenha o maior valor**.

PROBLEMA DA MOCHILA - ALGORITMO

1. Listar todos os candidatos no passo atual;
 2. Usando o critério guloso, selecionar o melhor local;
 3. Adicionar o melhor local na solução e retirá-lo do conjunto de candidatos;
 4. Repetir até chegar a uma solução.
-

PROBLEMA DA MOCHILA - ALGORITMO

1. Listar todos os candidatos no passo atual;
2. Usando o critério guloso, selecionar o melhor local;
3. Adicionar o melhor local na solução e retirá-lo do conjunto de candidatos;
4. Repetir até chegar a uma solução.

-
1. **Listar todos os itens ainda não escolhidos (candidatos) no passo atual**, ordenando-os pelo critério de **valor**;
 2. **Usando o critério guloso, selecionar o melhor item** (o de maior valor) que respeite o limite de capacidade restante da mochila;
 3. **Adicionar o item selecionado na mochila (solução) e retirá-lo do conjunto** de itens disponíveis;
 4. **Repetir o processo até chegar a uma solução** (quando a mochila estiver cheia ou não restarem mais itens).

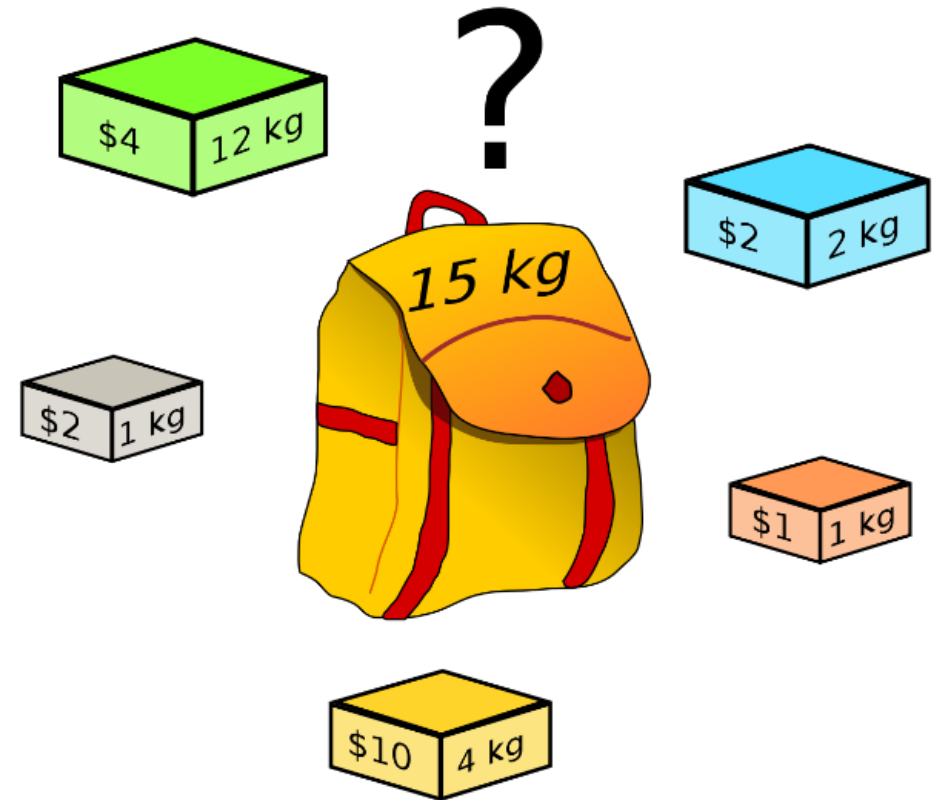
MOCHILA: FORÇA BRUTA

Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

Subconjunto	Peso Total	Valor Total
1,2,3,4	19	17
1,2,3,5	16	9
1,2,4,5	19	18
1,3,4,5	18	17
2,3,4,5	8	15
1,2,3,4,5	20	19

A MOCHILA

Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

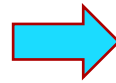


1) ORDENANDO POR VALOR

Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

1) ORDENANDO POR VALOR

Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2



Item	Peso	Valor
4	4	10
1	12	4
5	1	2
2	2	2
3	1	1

SELEÇÃO DO MELHOR LOCAL

Item	Peso	Valor
4	4	10
1	12	4
5	1	2
2	2	2
3	1	1

Mochila:

Valor:

Restante:

SELEÇÃO DO MELHOR LOCAL

Item	Peso	Valor
4	4	10
1	12	4
5	1	2
2	2	2
3	1	1

Mochila: {}

Valor: 0

Restante: 15

SELEÇÃO DO MELHOR LOCAL

Item	Peso	Valor
4	4	10
1	12	4
5	1	2
2	2	2
3	1	1

Mochila: {}

Valor: 0

Restante: 15

SELEÇÃO DO MELHOR LOCAL

Item	Peso	Valor
4	4	10
1	12	4
5	1	2
2	2	2
3	1	1

Mochila: {4}

Valor: 10

Restante: 11

SELEÇÃO DO MELHOR LOCAL

Item	Peso	Valor
4	4	10
1	12	4
5	1	2
2	2	2
3	1	1

Mochila: {4}

Valor: 10

Restante: 11

SELEÇÃO DO MELHOR LOCAL

Item	Peso	Valor
4	4	10
1	12	4
5	1	2
2	2	2
3	1	1

Mochila: {4}

Valor: 10

Restante: 11

SELEÇÃO DO MELHOR LOCAL

Item	Peso	Valor
4	4	10
1	12	4
5	1	2
2	2	2
3	1	1

Mochila: {4}

Valor: 10

Restante: 11

SELEÇÃO DO MELHOR LOCAL

Item	Peso	Valor
4	4	10
1	12	4
5	1	2
2	2	2
3	1	1

Mochila: {4}

Valor: 10

Restante: 11

SELEÇÃO DO MELHOR LOCAL

Item	Peso	Valor
4	4	10
1	12	4
5	1	2
2	2	2
3	1	1

Mochila: {4,5}

Valor: 12

Restante: 10

SELEÇÃO DO MELHOR LOCAL

Item	Peso	Valor
4	4	10
1	12	4
5	1	2
2	2	2
3	1	1

Mochila: {4,5}

Valor: 12

Restante: 10

SELEÇÃO DO MELHOR LOCAL

Item	Peso	Valor
4	4	10
1	12	4
5	1	2
2	2	2
3	1	1

Mochila: {4,5}

Valor: 12

Restante: 10

SELEÇÃO DO MELHOR LOCAL

Item	Peso	Valor
4	4	10
1	12	4
5	1	2
2	2	2
3	1	1

Mochila: {4,5,2}

Valor: 14

Restante: 8

SELEÇÃO DO MELHOR LOCAL

Item	Peso	Valor
4	4	10
1	12	4
5	1	2
2	2	2
3	1	1

Mochila: {4,5,2}

Valor: 14

Restante: 8

SELEÇÃO DO MELHOR LOCAL

Item	Peso	Valor
4	4	10
1	12	4
5	1	2
2	2	2
3	1	1

Mochila: {4,5,2,3}

Valor: 15

Restante: 7

FORÇA BRUTA X GULOSO

Força Bruta:

Subconjunto	Peso Total	Valor Total
1,2,3,4	19	17
1,2,3,5	16	9
1,2,4,5	19	18
1,3,4,5	18	17
2,3,4,5	8	15
1,2,3,4,5	20	19

Guloso:

Mochila: {4,5,2,3}

Valor: 15

Restante: 7

FORÇA BRUTA X GULOSO



Força Bruta:

Subconjunto	Peso Total	Valor Total
1,2,3,4	19	17
1,2,3,5	16	9
1,2,4,5	19	18
1,3,4,5	18	17
2,3,4,5	8	15
1,2,3,4,5	20	19

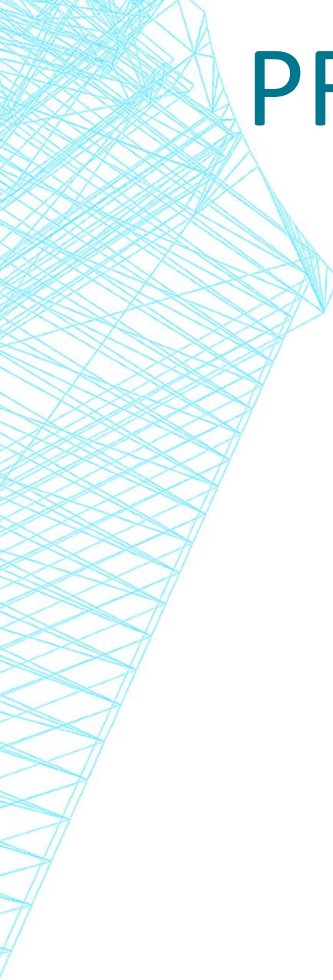
Guloso:

Mochila: {4,5,2,3}

Valor: 15

Restante: 7

PROBLEMA DA MOCHILA, EXEMPLO 2



Item	Peso	Valor
4	4	10
1	11	4
5	1	2
2	2	2
3	1	1

PROBLEMA DA MOCHILA, EXEMPLO 2

Item	Peso	Valor
4	4	10
1	11	4
5	1	2
2	2	2
3	1	1

Mochila:

Valor:

Restante: 15

PROBLEMA DA MOCHILA, EXEMPLO 2

Item	Peso	Valor
4	4	10
1	11	4
5	1	2
2	2	2
3	1	1

Mochila: {}

Valor: 0

Restante: 15

PROBLEMA DA MOCHILA, EXEMPLO 2

Item	Peso	Valor
4	4	10
1	11	4
5	1	2
2	2	2
3	1	1

Mochila: {4}

Valor: 10

Restante: 11

PROBLEMA DA MOCHILA, EXEMPLO 2

Item	Peso	Valor
4	4	10
1	11	4
5	1	2
2	2	2
3	1	1

Mochila: {4}

Valor: 10

Restante: 11

PROBLEMA DA MOCHILA, EXEMPLO 2

Item	Peso	Valor
4	4	10
1	11	4
5	1	2
2	2	2
3	1	1

Mochila: {4,1}

Valor: 14

Restante: 0

PROBLEMA DA MOCHILA, EXEMPLO 2

Item	Peso	Valor
4	4	10
1	11	4
5	1	2
2	2	2
3	1	1

Mochila: {4,1}

Valor: 14

A mochila está cheia!

Restante: 0

FORÇA BRUTA X GULOSO

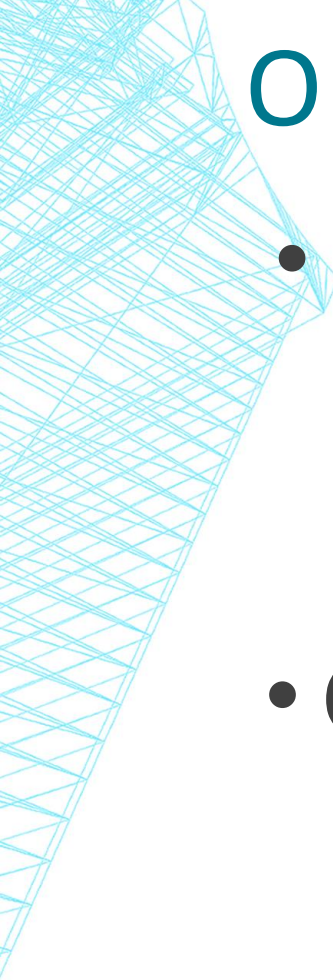


Subconjunto	Peso Total	Valor Total
1,2,3,4	19	17
1,2,3,5	16	9
1,2,4,5	19	18
1,3,4,5	18	17
2,3,4,5	8	15
1,2,3,4,5	20	19

Mochila: {4,1}

Valor: 14

Restante: 0



O PROBLEMA DA MOCHILA (*KNAPSACK PROBLEM*)

- Quem errou? O algoritmo ou quem projetou?
 - Desconsiderou-se a variável “*peso*”.
- Critério guloso, **alternativa 2**:
 - selecionar o item individual ainda não escolhido que caiba na mochila e tenha a maior **relação valor/peso**.

MOCHILA - EXEMPLO 2, OUTRO CRITÉRIO

Item	Peso	Valor	V/P
1	11	4	0.36
2	2	2	1
3	1	1	1
4	4	10	2.5
5	1	2	2

MOCHILA - EXEMPLO 2, OUTRO CRITÉRIO

Item	Peso	Valor	V/P
4	4	10	2,5
5	1	2	2
3	1	1	1
2	2	2	1
1	11	4	0,36

MOCHILA - EXEMPLO 2, OUTRO CRITÉRIO

Item	Peso	Valor	V/P
4	4	10	2,5
5	1	2	2
3	1	1	1
2	2	2	1
1	11	4	0,36

Mochila: {}

Valor: 0

Restante: 15

MOCHILA - EXEMPLO 2, OUTRO CRITÉRIO

Item	Peso	Valor	V/P
4	4	10	2,5
5	1	2	2
3	1	1	1
2	2	2	1
1	11	4	0,36

Mochila: {}

Valor: 0

Restante: 15

MOCHILA - EXEMPLO 2, OUTRO CRITÉRIO

Item	Peso	Valor	V/P
4	4	10	2,5
5	1	2	2
3	1	1	1
2	2	2	1
1	11	4	0,36

Mochila: {4}

Valor: 10

Restante: 11

MOCHILA - EXEMPLO 2, OUTRO CRITÉRIO

Item	Peso	Valor	V/P
4	4	10	2,5
5	1	2	2
3	1	1	1
2	2	2	1
1	11	4	0,36

Mochila: {4}

Valor: 10

Restante: 11

MOCHILA - EXEMPLO 2, OUTRO CRITÉRIO

Item	Peso	Valor	V/P
4	4	10	2,5
5	1	2	2
3	1	1	1
2	2	2	1
1	11	4	0,36

Mochila: {4,5}

Valor: 12

Restante: 10

MOCHILA - EXEMPLO 2, OUTRO CRITÉRIO

Item	Peso	Valor	V/P
4	4	10	2,5
5	1	2	2
3	1	1	1
2	2	2	1
1	11	4	0,36

Mochila: {4,5}

Valor: 12

Restante: 10

MOCHILA - EXEMPLO 2, OUTRO CRITÉRIO

Item	Peso	Valor	V/P
4	4	10	2,5
5	1	2	2
3	1	1	1
2	2	2	1
1	11	4	0,36

Mochila: {4,5,3}

Valor: 13

Restante: 9

MOCHILA - EXEMPLO 2, OUTRO CRITÉRIO

Item	Peso	Valor	V/P
4	4	10	2,5
5	1	2	2
3	1	1	1
2	2	2	1
1	11	4	0,36

Mochila: {4,5,3}

Valor: 13

Restante: 9

MOCHILA - EXEMPLO 2, OUTRO CRITÉRIO

Item	Peso	Valor	V/P
4	4	10	2,5
5	1	2	2
3	1	1	1
2	2	2	1
1	11	4	0,36

Mochila: {4,5,3,2}

Valor: 15

Restante: 7

FORÇA BRUTA X GULOSO



Subconjunto	Peso Total	Valor Total
1,2,3,4	19	17
1,2,3,5	16	9
1,2,4,5	19	18
1,3,4,5	18	17
2,3,4,5	8	15
1,2,3,4,5	20	19

Mochila: {4,5,3,2}

Valor: 15

Restante: 7

MAS ... SUBESTRUTURA ÓTIMA?

Item	Peso	Valor	V/P
1	10	60	6
2	20	100	5
3	30	120	4

Capacidade = 50

Mochila:

Valor:

Restante:

MAS ... SUBESTRUTURA ÓTIMA?

Item	Peso	Valor	V/P
1	10	60	6
2	20	100	5
3	30	120	4

Capacidade = 50

Mochila:

Valor:

Restante:

MAS ... SUBESTRUTURA ÓTIMA?

Item	Peso	Valor	V/P
1	10	60	6
2	20	100	5
3	30	120	4

Capacidade = 50

Mochila: {1}

Valor: 60

Restante: 40

MAS ... SUBESTRUTURA ÓTIMA?

Item	Peso	Valor	V/P
1	10	60	6
2	20	100	5
3	30	120	4

Capacidade = 50

Mochila: {1}

Valor: 60

Restante: 40

MAS ... SUBESTRUTURA ÓTIMA?

Item	Peso	Valor	V/P
1	10	60	6
2	20	100	5
3	30	120	4

Capacidade = 50

Mochila: {1,2}

Valor: 160

Restante: 20

MAS ... SUBESTRUTURA ÓTIMA?

Item	Peso	Valor	V/P
1	10	60	6
2	20	100	5
3	30	120	4

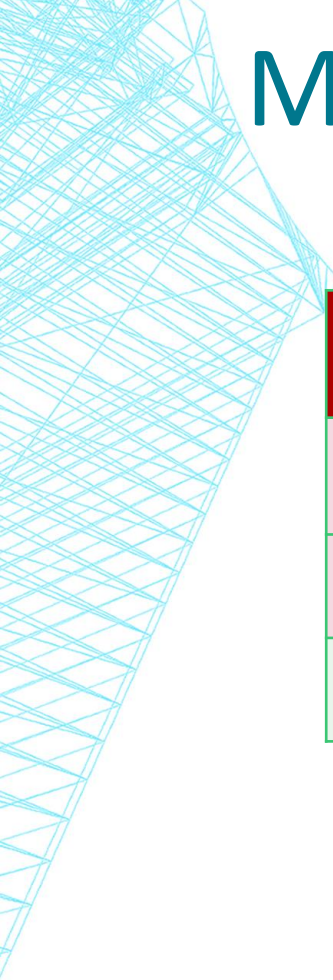
Capacidade = 50

Mochila: {1,2}

Valor: 160

Restante: 20

MAS ... SUBESTRUTURA ÓTIMA?



Item	Peso	Valor	V/P
1	10	60	6
2	20	100	5
3	30	120	4

Mochila: {1,2}

Valor: 160

Restante: 20

No entanto

Mochila: {2,3}

Valor: 220

Restante: 0

O CONTEXTO: A MOCHILA 0-1 (INTEIRA)

- **O que vínhamos fazendo até aqui?**
 - Estávamos trabalhando com o cenário da Mochila 0-1, onde os itens são indivisíveis. Ou levamos o item inteiro (1) ou o deixamos para trás (0). Não existe "meio termo".
- **A Falha do Algoritmo Guloso:**
 - Seguindo a razão Valor/Peso (V/P), o algoritmo escolheu o **Item 1** (razão 6).
 - Essa escolha precoce consumiu espaço e bloqueou a entrada conjunta dos **Itens 2 e 3**.
 - **Resultado Guloso:** Item 1 + Item 2 = Valor 160 (sobram 20kg vazios).
 - **Solução Ótima Real:** Item 2 + Item 3 = Valor 220 (mochila cheia). A Grande



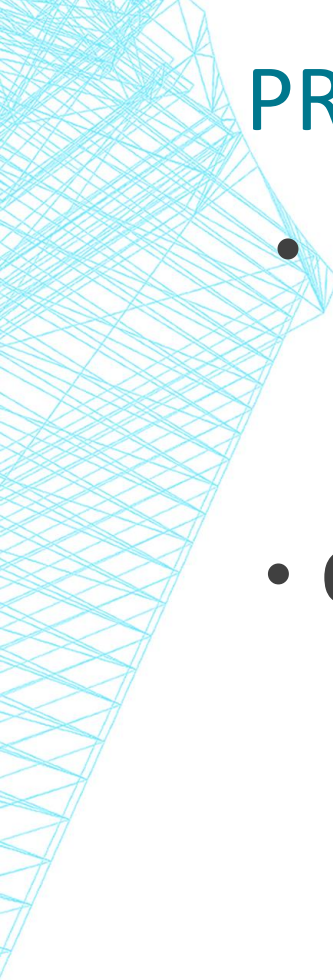
O PROBLEMA DA MOCHILA (*KNAPSACK PROBLEM*)

A grande pergunta: Se o algoritmo guloso falhou, significa que este problema não tem Subestrutura Ótima?

O PROBLEMA DA MOCHILA *(KNAPSACK PROBLEM)*

A grande pergunta: Se o algoritmo guloso falhou, significa que este problema não tem Subestrutura Ótima?

- Resposta: **Não!** Ele possui subestrutura ótima. Vamos provar com os números.



PROVANDO A SUBESTRUTURA ÓTIMA NA MOCHILA 0-1

- **Relembrando a Definição:** *"Uma solução ótima para o problema contém em seu interior soluções ótimas para subproblemas."*
- **O Problema Maior (Global):**
 - Capacidade total da mochila: **50**
 - Sabemos que a Solução Ótima Global é levar os **Itens 2 e 3** (Valor total: 220).

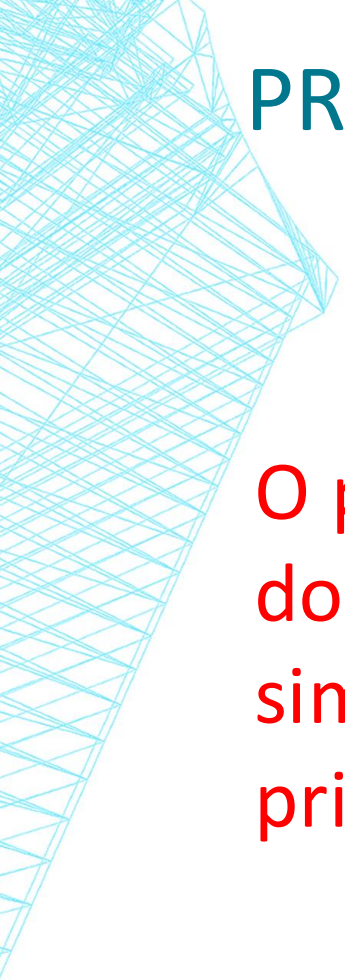
PROVANDO A SUBESTRUTURA ÓTIMA NA MOCHILA 0-1

- **Reduzindo para o Subproblema:**

- Imagine que decidimos levar o **Item 3** (Peso: 30, Valor: 120).
- O que sobra para resolver? Uma "mochila menor" com:
 - Nova capacidade: **20** ($50 - 30$)
 - Itens restantes disponíveis: **Item 1** e **Item 2**.

- **A Solução do Subproblema está contida no Todo:**

- Qual é a solução ótima absoluta para preencher essa mochila menor de capacidade 20? É levar o **Item 2** (Peso: 20, Valor: 100).
- **Conclusão:** A solução ótima do problema maior (Itens 2 + 3) contém, cirurgicamente em seu interior, a solução ótima do subproblema de capacidade 20 (Item 2).



PROVANDO A SUBESTRUTURA ÓTIMA NA MOCHILA 0-1

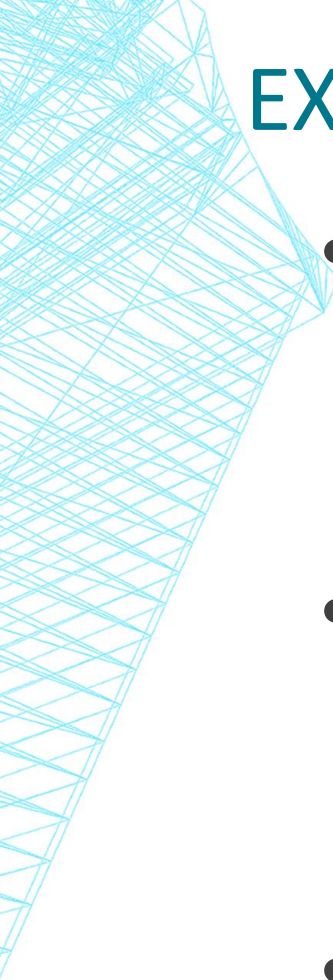
O problema da Mochila 0-1 **tem** subestrutura ótima. O motivo do algoritmo guloso falhar não é a falta de subestrutura, mas sim o fato de que escolhas locais míopes (como pegar o Item 1 primeiro) quebram o caminho para alcançar essa solução ótima.



ALGORITMOS GULOSOS: QUAL É O FOCO?

“One thing you will notice about greedy algorithms is that they are usually easy to design, easy to implement, easy to analyze, and they are very fast, but they are almost always difficult to prove correct.”

Ian Parberry



EXERCÍCIO 1

- Uma empresa precisa transportar diversas cargas em um caminhão com um caçamba de capacidade de 42m^3 .
- As cargas têm volumes de 16, 12, 13, 11, 10, 12, 9, 8, 15 e 14 m^3 e a empresa quer tentar fazer no máximo 3 viagens de transporte.
- Projete e execute um algoritmo guloso que resolva este problema

EXERCÍCIO 2

- Problema do troco (*coin change*)
 - Dado um sistema monetário S com um conjunto de moedas pré-definido, qual é a menor quantidade possível de moedas para retornar um troco C ?
 - A) $S = \{1, 5, 10, 25, 50, 100\}$
 $C = 147$
 - B) $S = \{1, 5, 15, 20, 50, 100\}$
 $C = 180$